

# Social Media Mining

simi

March 2022



# Contents



# Introduzione

L'obiettivo del corso è estrarre conoscenza da tutti i dati che vengono generati attraverso le piattaforme di social media. Verranno presentate tecniche di estrazione della conoscenza da un social media. Quindi centrale nel corso è l'idea di network (rete), maggior parte del programma è dedicata ai grafi. Confrontandone le proprietà e studiare le rete larg scale (social media) più dei studi che faremo più in piccolo per capire le connessioni tra le varie persone nella rete. Cominciamo precisi a .30 sempre.

Dobbiamo imparare ad analizzare un social media tramite:

- network science: è un termine introdotto 10 anni fa quando ci si è accorti che ogni sistema complesso ha delle caratteristiche delle rete
- machine learning

Tutti i metodi che vediamo sono applicabili a qualsiasi rete, noi vedremo l'applicazione social media. Una rete non è altro che un grafo, rivediamo quindi le definizioni principali per avere un linguaggio comune.

Fino agli inizi degli anni '70 quando si voleva studiare una rete sociale (offline) si utilizzava un approccio sociologico, che detto in parole povere, significa focalizzarsi sul singolo individuo. Si misurano le caratteristiche dei singoli individui e poi si aggregano le caratteristiche. Già nel 1930 si cominciava a pensare che se si voleva capire qualcosa di più di una società non bisognava solo capire i singoli individui ma anche come si connetevano tra di loro, nacque la sociometria. Il primo studio di network analysis, in cui hanno studiato una rete di relazioni, è stato Moreno, che è diventato il nome del fondatore della network science. Prima la rete la chiamavano sociogramma, individui identificati come nodi e se avevano una relazione collegati da un link, quindi un grafo. Moreno cominciò a parlare di analisi di rete sociali, che ha diverse caratteristiche:

- Intuizione strutturale, capire com'è fatta la rete mi aiuta a capire come sono le caratteristiche della società
- Empirica: l'approccio dev'essere o dei dati da cui costruisco il sociogramma, visualizzo il sociogramma (ora non ha più senso visualizzare, reti troppo grandi)
- Metodi matematici
- Metodi computazionali

Da allora il focus dello studio delle grandi popolazioni si è spostato da singolo individuo a rete. Per esempio il legame di relazione tra gli attori della società sono più importanti degli attributi. Invece che focalizzarsi sugli attributi di un singolo individuo vede le caratteristiche sociali come derivano dalle relazioni strutturali. Questo si collega alle analytics dei social media, perchè ci danno delle percentuali, non ci danno le informazioni di com'è fatta la rete. Sarà nostro compito capire la rete data la struttura delle relazioni di un social network.

## Social Network Analysis

Ha due significati:

- Studio di una social network
- La metodologia per studiare la social network, potrebbe quindi essere un sinonimo di network science

## Complex Sistem

Si sono accorti che la maggior parte dei sistemi sono sistemi complessi, ci siamo accorti che la maggior parte dei sistemi interessanti sono sistemi complessi. Un sistema complesso è un sistema in cui il comportamento del sistema non può essere spiegato come aggregazione dei comportamenti delle singole parti, E' un sistema in cui solo capendo la rete delle relazioni che sta intorno ai nodi si riesce a spiegare il comportamento del sistema. Ditemi qualunque sistema complesso c'è una rete, parliamo di secolo della complessità perchè se pensiamo ai grandi sistemi di cui parliamo (cervello, finanza, economia, trasporti, social network) sono tutti sistemi complessi, perchè se ne studio le singole parti non ne capisco il comportamento.

Il problema è che se voglio capire un sistema complesso e voglio fare una rete, devo averne i dati. Avere i dati delle relazioni tra individui è complicato.

Come dicevamo ogni sistema complesso è una rete. Quello che si è visto studiando i sistemi complessi con analisi di reti si è visto che avevano caratteristiche simili, si potevano analizzare le reti con gli stessi sistemi anche se venivano da sistemi completamente diversi (social media e cervello). E' per questo che esiste la scienza delle reti. Quindi le reti sono molto più simili tra loro rispetto a quanto si pensi.

Se le reti sono grafi, ed i grafi li conosciamo da secoli, perchè abbiamo scoperto la network science solo adesso? Inanzitutto perchè per costruire la rete abbiamo bisogno di dati, e per capire un sistema complesso abbiamo bisogno di tanti dati, prima di questo millennio questa grande quantità di dati non c'era. Soltanto quando si è arrivati ai grandi sistemi sul web si è riuscito ad avere dati abbastanza grandi per riuscire a costruire mappe di sistemi complessi per studiarli. La teoria dei grafi per come è stata pensata dal punto di vista matematico copre una parte della struttura delle reti, mancano metodi per coprire grossi reti ed i pattern di complessità non interessanti per grandi reti.

La prima rete che si è riusciti a mappare è stata nel '98 una rete di attori e film, due attori avevano un link tra di loro se avevano partecipato allo stesso film. La rete presa dal web è stata nel '99 con il dominio .edu.

## Ripasso di Grafi

Cominciamo a definire cos'è una rete: è un insieme finito di attori (in sociologia) /nodi (network science)/vertici (grafi) e delle relazioni, che chiamiamo link (in

network science) e edge (nei grafi) che vengono definiti dagli attori del sistema. Quando studiamo una rete non sono importanti i nodi ma i link.

Il primo punto di studio di un social media è capire come rappresentarlo come un rete, questo sembra banale ma è tutt'altro, è il punto cruciale da cui partire in un progetto. Dobbiamo capire quali sono gli attori, quali relazioni tra attori vogliamo considerare. Ad esempio in un social media quali nodi considero? quelli che hanno fatto tot attività o hanno tot nodi? Quindi prima cosa è definire quali sono i nodi e poi filtrarli. Una volta definiti i nodi definire se ci sono delle operazioni di filtering da fare. Per esempio, quando si studiano le reti aziendali (si fa l'assessment di aziende sulla base del network) costruiamo la rete delle relazione delle persone che lavorano in azienda e vedo com'è fatta questa rete. Ad esempio se ci sono dei nodi centrali o se ci sono punti di rottura. Anche qui però bisogna capire cosa intendo come relazione tra persone, quindi anche sul link bisogna definire cosa intendiamo come relazione tra due nodi, dipende da due fattori:

- La domanda di ricerca e quale relazione risponde bene
- Quali dati ho a disposizione, questo è più difficile

Si possono fare filtri anche sui link, ad esempio se voglio una relazione di amicizia come definisco amicizia?

Quindi dato il sistema complesso, costruisco il design della rete, la definizione matematica di una rete è un grafo. Quindi quando parliamo di social media intendiamo:

- Il sistema complesso
- Il network
- Il grafo

In generale ci stiamo riferendo alla stessa cosa, però teniamo in mente che sono tre cose distinte, soprattutto rete e grafo vengono usate allo stesso modo.

Un grafo  $G$  è una coppia ordinata di Vertici e Link  $G = (V, E)$ , dove un link è una coppia di vertici. Abbiamo quindi un set di vertici  $V = v_1, v_2, \dots, v_n$  ed un set di edge  $E = e_1, e_2, \dots, e_n$  e un edge è una coppia di vertici  $e_k = (v_i, v_j)$ . Se ci viene chiesta una definizione di grafo di una rete, nei vertici vanno inclusi anche quelli isolati.

- SIZE: è la cardinalità del set di link della rete, ovvero il numero di link
- L'ORDER: è la cardinalità dei vertici, ovvero il numero dei nodi

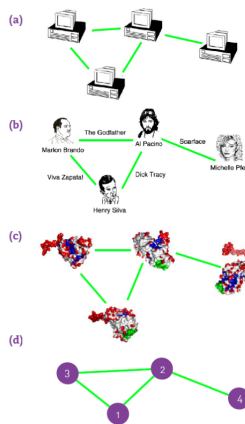
Dopo di che nell'uso comune si chiamano tutti e due network SIZE, sta a noi quando vediamo un articolo online capire la differenza (noi li useremo nel modo corretto però).

## Grafo non orientato

Il link è una coppia di vertici senza ordine, significa che non ho informazione sulla direzione delle relazioni. Quando parlo di link, poichè non so l'ordine, la coppia è non orientata, quindi il link  $(n_i, n_j) == (n_j, n_i)$ .

## Grafo orientato

Qui i link sono orientati, cioè la relazione parte dal primo nodo ed arriva al secondo, ad esempio il primo chiama il secondo. Quindi in questo caso la coppia di link  $(n_i, n_j) \neq (n_j, n_i)$  rappresenta due link diversi tra di loro.



## Rappresentazione Grafi

La rete si visualizza come i grafi, quindi un pallino per i nodi ed un link orientato o meno. Come visualizzo un grafo può essere visualizzato in n modi diversi, quindi la visualizzazione di un grafo non è univoca. Questo vuol dire che sistemi complessi molto diversi possono avere come rappresentazione lo stesso grafo, vuol dire che applico il metodo di network science che vale per tutti i grafi, è solo l'interpretazione che cambia.

## Rappresentazione matematica di un grafo

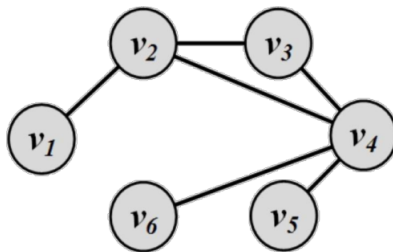
Noi facciamo le rappresentazioni classiche, non stiamo parlando di rappresentazione efficiente (ad esempio come rappresentare in maniera efficiente le matrici). Ci sono tre modi per rappresentare una rete come un grafo:

- Edge List
- Lista di adiacenza
- Matrice di adiacenza

**Edge List**

Faccio due liste:

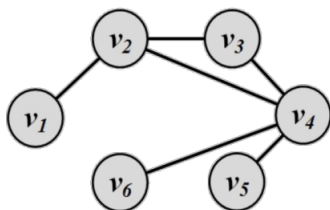
- Lista dei nodi
- Lista dei vertici



$(v_1, v_2)$   
 $(v_2, v_3)$   
 $(v_2, v_4)$   
 $(v_3, v_4)$   
 $(v_4, v_5)$   
 $(v_4, v_6)$

**Lista di adiacenza**

Elenca tutti i nodi e per ogni nodo elenca i suoi vicini, ovvero tutti i nodi che hanno un link con il nodo.

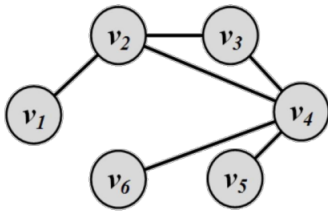


| Node  | Connected To         |
|-------|----------------------|
| $v_1$ | $v_2$                |
| $v_2$ | $v_1, v_3, v_4$      |
| $v_3$ | $v_2, v_4$           |
| $v_4$ | $v_2, v_3, v_5, v_6$ |
| $v_5$ | $v_4$                |
| $v_6$ | $v_4$                |

**Matrice di adiacenza**

Costruisco una matrice  $N \times N$ , in cui metto 1 se esiste un link tra i due nodi o 0 altrimenti. La matrice di adiacenza è un modo comodo per rappresentare le reti.

$$A_{ij} = \begin{cases} 1, & \text{if there is a link between nodes } v_i \text{ and } v_j \\ 0, & \text{otherwise} \end{cases}$$

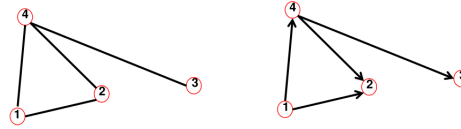


|       | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|-------|-------|-------|-------|-------|-------|-------|
| $v_1$ | 0     | 1     | 0     | 0     | 0     | 0     |
| $v_2$ | 1     | 0     | 1     | 1     | 0     | 0     |
| $v_3$ | 0     | 1     | 0     | 1     | 0     | 0     |
| $v_4$ | 0     | 1     | 1     | 0     | 1     | 1     |
| $v_5$ | 0     | 0     | 0     | 1     | 0     | 0     |
| $v_6$ | 0     | 0     | 0     | 1     | 0     | 0     |

### Grafi orientati

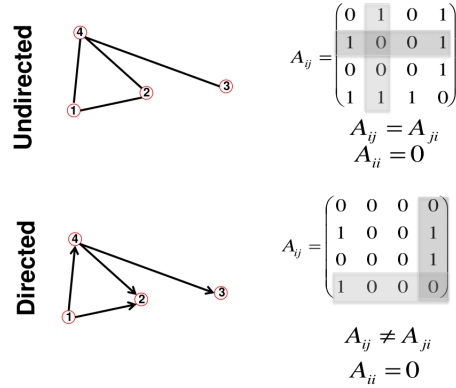
Se la rete orientata non rimane tutto uguale, perchè se la rete è orientata la matrice è simmetrica, se non è orientata non è simmetrica, dal punto di vista computazione è bello averle simmetriche perchè possiamo studiarne metà. Le matrici vanno bene per una rappresentazione matematica e quando la matrice è caricabile, quando parliamo di grandi reti non è facile caricare matrici troppo grandi.

Come definiamo una matrice di adiacenza per un grafo orientato, il link è 1 se il link parte dal secondo nodo ed arriva al primo, questo perchè andando per righe vengono meglio i conti matematici. Nell'esempio sul nodo 4 abbiamo un 1 in corrispondenza del nodo 1 perchè abbiamo un link che entra in 4 da 1.



$$A_{ij} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \quad A_{ij} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Vediamo un'interpretazione più facile, le righe danno gli in-link di un nodo, le colonne gli out-link.



### Loop

Un loop è un link tra un nodo e se stesso. I loop in tutti i dataset esistono, anche nei social media in cui non dovrebbero esserci, quindi la prima cosa da fare nel dataset è controllare se ci sono loop, se ci sono vanno cancellati in quanto tutta la teoria che vedremo è basata su reti senza loop. L'altra cosa che non

consideriamo è la possibilità è che tra due nodi ci siano più link. In una rete orientata posso avere tra due nodi due archi (a,b e b,a).

Quindi noi considereremo reti senza loop, senza multi edge, orientate e non orientate.

## Reti pesate

Una rete pesata è un grafo semplice in cui metto un etichetta sul link, questa etichetta è un numero (peso) che rende conto di una caratteristica che mi interessa. Nell'esempio vogliamo costruire una rete degli aeroporti, potrei mettere dei pesi ai link (la tratta) ad esempio il tempo, quante compagnie fanno la tratta. Nei social media è la forza della relazione. Ad esempio potrei contare quanti messaggi si scambiano su quel link.

La matrice di adiacenza delle reti pesate invece che l'1 metto il peso del link, 0 altrimenti.

## Altri tipi di reti

- Bipartite
- Multilayer: reti in cui ho più edge tra due nodi e ne faccio una modellazione come se fossero tante reti diverse ma collegate
- Attributed: ho informazioni in più sui nodi e sui link (ad esempio che lingua usano nei post)
- Temporal: studiamo l'evoluzione temporale della rete

## Affiliation Network

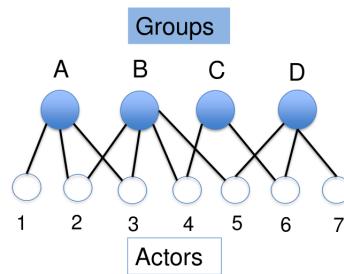
Vediamo un caso particolare di rete, che dal punto di vista della teoria non sono interessanti ma che capitano spesso nei social media. Dal punto di vista modellistico si chiamano reti bipartite. Le Affiliation Network hanno due tipi di nodi, i nodi hanno un valore diverso nella rete:

- Attori
- Gruppi

Per fare un esempio pensiamo ai canali YouTube che hanno i follower, in una rete in cui abbiamo sia canali che follower i nodi hanno valore diverso, si chiamano Affiliation perchè i follower sono affiliati ai canali. Capita spesso nei social media perchè c'è stato un cambio di paradigma, perchè il servizio che gli utenti vogliono in un social è un servizio a gruppi.

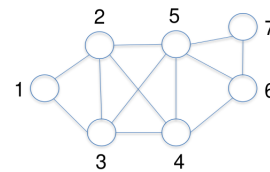
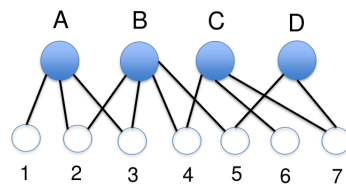
Questa condizione non basta a definire una Affiliation, ma anche i link devono esistere solo tra nodi diversi (non posso avere link tra gruppi ed attori), il link tiene conto del fatto che l'attore è affiliato a un gruppo. Se pensiamo agli utenti di un social media, quello che posso dire è che gli utenti hanno link tra di

loro perchè appartengono allo stesso gruppo, ma è una relazione implicita. La rappresentazione è simile alla matrice di adiacenza, è 1 se il nodo  $j$  appartiene al gruppo  $i$ , 0 altrimenti.



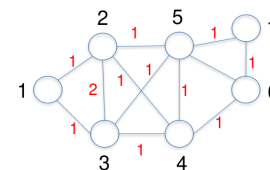
Il problema è che tutti i metodi di network science non sono costruiti per reti di questo tipo, dobbiamo quindi trasformare l'affiliation network in una rete standard. Lo facciamo con le proiezioni, cerchiamo di proiettare un affiliation network in una rete semplice.

- Proiezioni sui gruppi: ottengo una rete i cui nodi sono i gruppi e due gruppi sono connessi da un link se hanno in comune almeno un utente. Mi interessa quando voglio studiare le relazioni tra i gruppi. Ad esempio se voglio studiare se i canali YouTube hanno utenti in comune.
- Proiezione sugli utenti: considero una rete i cui nodi sono gli attori, due nodi sono collegati tra un link se gli attori sono affiliati almeno ad un gruppo. Gli utenti sono connessi agli stessi canali.

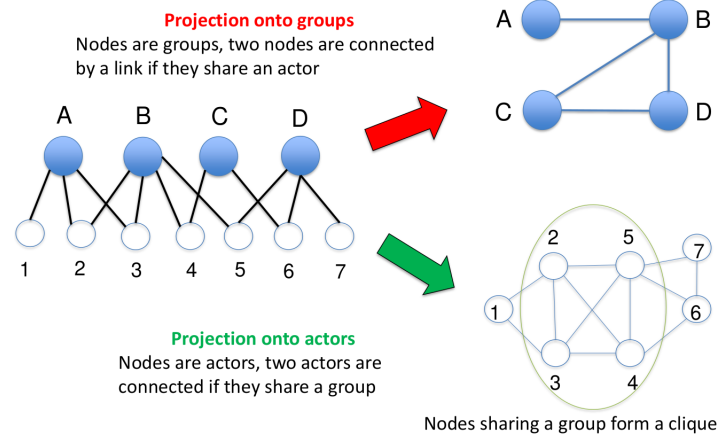


Information loss:  
how many groups two nodes share

Projection weighted: give each link a weight equal to the number of common groups for example



L'operazione non è regressibile perchè non abbiamo il peso sui link, quindi per rendere reversibile l'operazione peso il link con il numero di gruppi che ho in condivisione.



C'è un famosissimo studio, human disease network. Si è costruita una rete che mettesse in risalto la relazione delle malattie data dalla genetica, due malattie sono connesse perchè hanno la mutazione dallo stesso gene. Chiaramente questa è una rete bipartita, perchè non metto in relazione due malattie direttamente, ma una malattia fa parte di un gruppo. Dopo di che posso proiettare sulle malattie, due malattie sono connesse perchè espresse dalla stessa mutazione di geni, oppure proietto sui geni e dico che due geni hanno un link perchè una mutazione porta alla stessa malattia. Facendo community detection si è visto quali gruppi di malattie erano simili, perchè erano espressione della stessa mutazione genetica.

## Reti Tripartite

Possono esistere anche reti n-partite. L'esempio è quello delle ricette, gli ingredienti e i composti chimici degli ingredienti. Un ingrediente può partecipare a più ricette, così come un composto chimico può partecipare a più ingredienti. Da questo studio si è visto che gli stessi composti chimici partecipano a ricette molto diverse.

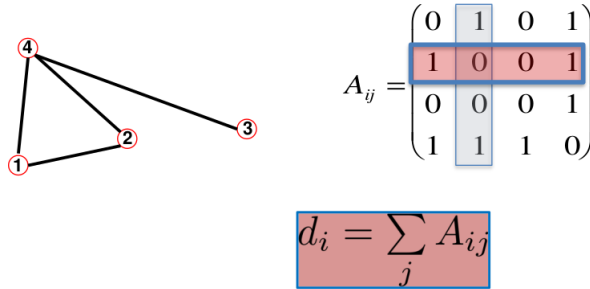
## Densità della rete

E' una proprietà globale della rete e va calcolata in qualunque studio, perchè ci da un'idea di com'è fatta la rete.

### Reti non orientate

#### Grado dei nodi

E' una delle proprietà fondamentali delle reti, si chiama il grado del nodo il numero di link connessi a quel nodo, in termini di social media, quanti vicini (amici) ha quel nodo. Tipicamente si usa come notazione  $d_i$  o  $k_i$  del nodo  $i$  (d come degree). Il numero di connessioni di un nodo del social media misura l'importanza del nodo all'interno della rete. Supponiamo di avere per un grafo  $G(N, L)$  un set di nodi  $N$  e il set di link  $L$ . Possiamo quindi scrivere la matrice di adiacenza del grafo. Come facciamo a calcolare il degree di un nodo a partire dalla matrice di adiacenza? Il grado di un nodo  $i$  è la sommatoria su  $j$  della riga della matrice di adiacenza.



Un grafo è un insieme di nodi, su ogni nodo posso calcolare delle caratteristiche, cosa ne faccio dopo? faccio una distribuzione di quella caratteristica, se non ne voglio fare un'intera distribuzione ne calcolo almeno il valore medio (voglio sapere in media i nodi di una rete quante connessioni hanno). Si chiama average degree la media di numeri di link dei nodi. Come per qualsiasi media sommo il grado dei nodi di tutti i nodi diviso il numero di nodi.

$$\langle d \rangle = \frac{\sum_{i=1}^N d_i}{N}$$

In un social network la vera ricchezza sono i link dei nodi, quindi il grado medio dei nodi è una misura molto importante da dare, indica in media un nodo a quanti altri nodi si connette.

Queste misure implicano la conoscenza della struttura del grafo, perchè dobbiamo sapere dove sono i link.

Se invece conosciamo il numero di nodi e di link possiamo comunque avere un'informazione sul grado dei nodi? Possiamo avere quello medio.

$$\begin{aligned} \langle d \rangle &= \frac{\sum_{i=1}^N d_i}{N} \\ \sum_{i=1}^N d_i &= 2L \\ \langle d \rangle &= \frac{2L}{N} \end{aligned}$$

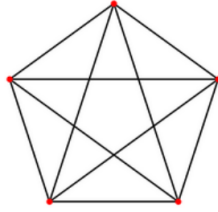
In questo caso il valore medio del nodo è 2 volte il numero dei link fratto il numero di nodi:

$$\langle d \rangle = \frac{2L}{N}$$

Una proprietà può essere solo locale o globale, ma quando considera una proprietà locale e ne faccio una media ottengo una proprietà locale mediata su tutta la rete, che è diversa da una proprietà globale (che è un numero che studia tutta la rete). Quindi il grado è una proprietà locale e anche se ne facciamo una distribuzione o una media rimane una proprietà locale.

### Densità

Se vogliamo avere una metrica globale che ci indica se una rete ha tanti link o pochi, è vero che il grado medio già risponde, ma cerchiamo una proprietà più globale. Dobbiamo calcolare la densità della rete come rapporto di  $L$  di link che la rete ha ed il numero massimo di link che la rete può avere, questa cosa si chiama grafo completo (matrice di adiacenza con tutti 1 tranne sulla diagonale).



Il grafo completo si calcola con la moltiplicazione tra i nodi  $N$  e gli altri nodi  $N-1$ , il tutto fratto due, per evitare di considerare ogni link due volte, è il coefficiente binomiale:

$$\frac{N(N-1)}{2}$$

quindi la densità di una rete è:

$$\Delta = \frac{L}{\frac{N(N-1)}{2}} = \frac{2L}{N(N-1)}$$

La differenza è che la densità va come il quadrato del numero di nodi ed il valore medio va come il numero di nodi:

$$\Delta = \frac{L}{\frac{N(N-1)}{2}} = \frac{2L}{N(N-1)}$$

$$\langle d \rangle = \frac{2L}{N}$$

L'aggiunta di un nodo in una rete drasticamente ne riduce la densità della rete.

La densità e il valore medio sono legate da una formula, la densità è il grado medio fratto N-1, dimostriamolo:

$$\langle d \rangle = \frac{2L}{N} \Delta = \frac{2L}{N(N-1)}$$

$$\Delta = \frac{\langle d \rangle}{N-1}$$

E' vero che entrambe tengono conto di quanti link ho nella rete, ma la densità tiene conto del fatto che il numero è legato al quadrato del numero di nodi.

## Reti orientate

Se siamo in una rete orientata dobbiamo ripartire a definire cos'è il grado di un nodo, perchè in questo caso abbiamo link uscenti ed entranti, che chiaramente danno informazioni completamente diverse. Il link uscente dice quanto il nodo si vuole esporre nella rete, un link entrante dice quanto un nodo è esposto. Quindi abbiamo in-degree  $(j, i)$  ed out-degree  $(i, j)$ .

Abbiamo già visto come si costruiscono le matrici di adiacenza dei grafi orientati, la riga dice quali sono i link entranti di un nodo e la colonna i link uscenti, calcolare gli out-degree sommo la colonna:

$$\sum_{i=1}^n A_{ij}$$

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

se invece voglio gli in-degree sommo sulla riga corrispondente al nodo:

$$\sum_{j=1}^n A_{ij}$$

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Supponiamo di aver calcolato per ogni nodo l'in-degree e l'out-degree e poi fare il valor medio di questi con la somma diviso il numero di nodi:

$$\langle d^{in} \rangle = \frac{\sum_{i=1}^N d_i^{in}}{N}$$

$$\langle d^{out} \rangle = \frac{\sum_{i=1}^N d_i^{out}}{N}$$

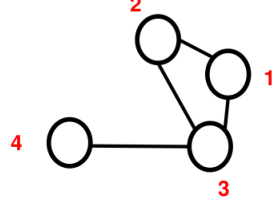
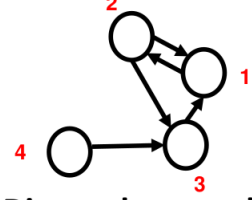
Sono uguali, perchè ogni link è uscente da un nodo ed entrante nell'altro e sono uguali a  $L$  fratto  $N$ , non c'è nessun due perchè il link essendo diverso in una direzione e nell'altra non viene contato due volte:

$$L = \sum_{i=1}^N d_i^{in} = \sum_{i=1}^N d_i^{out}$$

$$\langle d^{in} \rangle = \langle d^{out} \rangle = \frac{L}{N}$$

**Densità**

Come definizione rimane uguale, cambia solo come calcolarla:

**Undirected network****Directed network**

$$\langle d \rangle = \frac{2L}{N}$$

$$L_{max} = \frac{N(N-1)}{2}$$

$$\Delta = \frac{L}{N(N-1)/2} = \frac{2L}{N(N-1)}$$

$$\Delta = \frac{\langle d \rangle}{N-1}$$

$$\langle d^{in} \rangle = \langle d^{out} \rangle = \frac{L}{N}$$

$$L_{max} = N(N-1)$$

$$\Delta = \frac{L}{N(N-1)}$$

$$\Delta = \frac{\langle d^{in} \rangle}{N-1} = \frac{\langle d^{out} \rangle}{N-1}$$

## Le reti sono sparse

Nei social media si dice che le reti sono sparse, si intende reti di cui le matrici di adiacenza si dicono sparse, sono matrici con tanti zeri. Tipicamente nelle grandi reti ogni nodo, anche se fa tanti link, sono comunque molto minori a quelli che potrebbe fare  $(N-1)$ . Questo vuol dire che i valori delle densità sono molto molto piccoli, questi valori sono così bassi sono dati dal fatto che il valore va al quadrato del numero di nodi.

| Undirected network | N        | L        |
|--------------------|----------|----------|
| Internet           | 1.92E+05 | 6.09E+05 |
| Mobile phone calls | 3.66E+04 | 9.18E+04 |
| e-mail             | 5.72E+04 | 1.04E+05 |
| Actor network      | 7.02E+05 | 2.94E+07 |
| protein network    | 2018     | 2930     |
| Facebook2011       | 7.21E+08 | 6.90E+09 |
| Twitter2009        | 4.16E+07 | 1.4E+09  |
| Youtube            | 1.10E+06 | 2.90E+06 |

## Graph Densification

E' interessante vedere se un rete ha una densità che cresce o decresce nel tempo. Significa che vediamo un snapshot della densità di una rete nel tempo. Varia perchè entrano nuovi nodi, quindi potenzialmente ogni nodo che entra diminuisce la densità. Ma quello che si è visto nelle reti sociali è che man mano che si va avanti i nodi creano nuovi link rispetto ai nodi che entrano, si è visto quindi che la rete sociale si densifica, quindi in generale la densità della rete sociale aumenta (o rimane uguale). Capitava però che nelle reti grosse si creavano molti link senza valori (senza comunicazioni tra i nodi), per l'owner del social media è un costo un link tra utenti, si trovavano quindi un sacco di costi per mantenere link che non portavano valore (hanno anche fatto una campagna in cui si diceva di cancellare i link non utili).

La densità è un valore che ci dice immediatamente la tendenza dei nodi a formare connessioni, cosa che il grado medio non ci dice.



# Modelli di Rete

Useremo un metodo data driven, per induzione dai dati induco delle conclusioni sulle reti. Perché quindi abbiamo bisogno di modelli di rete?

- Capire come una proprietà, che magari misuro sulla rete, ha un impatto sul comportamento generale della rete che sto studiando. Ipotizziamo di aver studiato che il grado medio è 10, questa proprietà che misuro che effetti ha sulla rete stessa? un modello di rete mi permette di predire un comportamento globale a partire da una proprietà che misuro. Quindi dato un modello di rete ed una proprietà ne deduciamo le altre.
- Funzionano da null model. Supponiamo di avere la rete e abbiamo misurato qualcosa, a questo punto dobbiamo chiederci se quello che abbiamo trovato è interessante oppure anche una rete completamente random avrebbe questa caratteristica. Se abbiamo un modello di rete random e compariamo i risultati con un modello nullo capiamo se la caratteristica è significativa o meno (se i risultati sono diversi è significativa, altrimenti no).
- Ci permettono di capire e predire quali sono i meccanismi di crescita di una rete. Un modello di rete è un modello che permette di generare reti sintetiche (non vengono da dati reali), quando faccio questo faccio crescere la rete, se in questo modello di rete random ci metto qualche meccanismo posso capire come un determinato meccanismo ha impatto sulla crescita della rete.

Ogni modello di rete fissa dei parametri, mentre altri aspetti sono completamente random. Quindi chiamo modelli di rete dei modelli matematici in cui fisso dei parametri e lascio tutti gli altri aspetti randomici. Fissare dei parametri significa che fisso delle proprietà.

## Reti non orientate

Se pensiamo ad una rete possiamo pensare di fissare il numero di nodi, in tutti i modelli di rete random statici il numero di nodi è fissato. Il modello agisce nel decidere dove posizioniamo i link. Quando facciamo un modello random posizioniamo i link senza alcun meccanismo, ma li posizioniamo in modo completamente casuale. Esistono due modelli di rete random:

- Fisso il numero di nodi ed il numero di link, ho  $N$  nodi,  $L$  link a disposizione e posizioni questi  $L$  link in modo random.
- Fisso il numero di nodi, non fisso il numero di link totale, ma fisso la probabilità  $p$  che esista un link tra una coppia di nodi.

Entrambi i modelli sono completamente random, perché i link sono posizionati senza alcun meccanismo. Quale dei due utilizzare? Mentre il primo modello non ha dentro le probabilità, ma calcolare le proprietà è molto complicato. Il modello con la probabilità (avendo appunto una probabilità) permette

di avere un modello probabilistico permettendo di sfruttare la teoria delle probabilità calcolando analiticamente velocemente le probabilità.

Chiamiamo rete random una rete di  $N$  nodi, in cui ogni nodo è connesso con probabilità  $p$ .  $p$  è valido per ogni link. Non solo in questo modello i link sono messi a caso, ma otteniamo ad ogni run una rete con un numero di nodi diverso. Quindi questo modello di rete è complicato, bisogna stare attenti che ogni realizzazione, anche con gli stessi parametri, fa ottenere una rete diversa.

## Modello di Erdos-Renyi

Ma allora qual'è la distribuzione del numero totale di link, perchè ogni volta che abbiamo un modello probabilistico dobbiamo capire in media quanti link fa il modello, oltre alla media dobbiamo chiederci qual'è la deviazione standard ed ancora di più sappiamo che esistono le distribuzioni di probabilità.

Ogni volta in cui ho un processo in cui ho due possibili risultati (testa o croce) ed una probabilità  $p$  si chiama processo di Bernoulli, se ho più tentativi (successi) ho una distribuzione binomiale.

La probabilità che una realizzazione di un modello abbia un numero di link totali è questa:

$$P(L) = \binom{\binom{N}{2}}{L} p^L (1-p)^{\frac{N(N-1)}{2} - L}$$

Dati  $N$  nodi i possibili link che esistono con probabilità  $p$  sono  $n(n-1)/2$ , ogni link è un tentativo con una fissata probabilità ed ho un numero di tentativi, la distribuzione binomiale mi dice quindi qual'è la probabilità di avere un link nel modello. Quindi il modello segue una distribuzione binomiale.

Sappiamo che in una distribuzione binomiale il valor medio è il parametro  $p$  per il numero di tentativi (numero di link possibili):

$$\langle L \rangle = \sum_{L=0}^{\frac{N(N-1)}{2}} L P(L) = p \frac{N(N-1)}{2}$$

Posso anche calcolare la varianza, perchè lo sappiamo:

$$\sigma^2 = p(1-p) \frac{N(N-1)}{2}$$

Poi mi ricordo che il valor medio del grado è 2 volte il valor medio dei link su  $N$ :

$$\langle k \rangle = \frac{2 \langle L \rangle}{N} = p(N-1)$$

Il modello è potente, perchè una volta riconosciuto che è un binomiale non dobbiamo più calcolare nulla, è tutto già fatto.

Quindi riassumendo, il modello di rete random fissa il numero di nodi ed il numero di probabilità di esistenza di un link. Bello perchè essendo un modello di

Bernulli posso calcolare facilmente tutte le caratteristiche, come il valor medio, la varianza.

Utilizziamo i modelli nulli con la funzione di comparare quello che osservo in una rete sociale reale con una rete random (che deve avere caratteristiche simili). Se sto analizzando una rete sociale con  $N$  nodi utilizzerò una rete random con  $N$  nodi. Ma con che probabilità  $p$  facciamo costruire i link? Posso richiedere che il valore del numero totale dei link della mia rete sia uguale al valor medio del numero di link della rete random. Imponendo questa condizione, sostituendo nella formula di prima, posso trovare  $p$ :

$$\begin{aligned}\Delta &= \frac{L}{N(N-1)/2} = \frac{2L}{N(N-1)} \\ L &= \Delta N(N-1)/2 \\ \langle L \rangle &= \sum_{\frac{N(N-1)}{2}}^{L=0} = p \frac{N(N-1)}{2} \\ p &= \Delta\end{aligned}$$

Costruisco la rete random imponendo che il parametro  $p$  sia uguale alla rete sociale che sto studiando. E' ovvio, perchè la densità è la probabilità di un nodo di creare un link.

Quindi la differenza tra una rete sociale reale ed una rete random è la posizione dei nodi, se sono effettivamente diversi allora stiamo studiando una rete con delle caratteristiche significative.

## Reti orientate

Nelle reti orientate è tutto uguale, perchè il concetto di densità è uguale. Il parametro  $p$  è ancora uguale alla densità.

## Degree Distribution

La prima proprietà che si studia in una rete è il grado dei nodi (influencer). La prima quantità interessante da studiare in una rete è la distribuzione del grado dei nodi, ricordiamo che il grado di un nodo è il numero di connessioni che ha, è importante perchè si dice misura di centralità del nodo. Un nodo è tanto più centrale (importante), tanto più il suo grado è elevato (tanti più follower abbiamo tanto siamo influenti). Andiamo a vedere se il grado dei nodi di una rete reale è diverso da quello che si otterrebbe in una rete random. Significa che sotto c'è qualcosa che fa crescere la rete non in modo randomico.

Misuro per ogni nodo quante connessioni ha e poi posso fare la media, la deviazione standard... in generale studio la distribuzione del grado dei nodi. Lo facciamo prima sulle reti random e poi lo faremo sulle reti sociali.

Senza fare la distribuzione diciamo che la distribuzione del grado in una rete random è binomiale, perchè ogni nodo ha  $n-1$  tentativi di fare link con gli

altri  $n-1$  nodi ed ogni link ha probabilità  $p$  di esserci, di nuovo è un processo Bernulliano, quante prove di Bernulli faccio?  $N-1$ . Quindi di nuovo siamo in caso di una distribuzione binomiale con parametri  $N-1$  e  $p$ .

$$P(k) = \binom{N-1}{k} p^k (1-p)^{(N-1)-k}$$

Conosco quindi a priori qual'è la distribuzione del grado dei nodi in una rete random, è binomiale quindi il valor medio del grado è  $\langle k \rangle = p \cdot N - 1$ . Quello che è interessante è dividere la deviazione standard per il valor medio. La varianza (deviazione standard dalla media) ci dice quanto sono lontani i valori dal valore medio, la statistica del mezzo pollo, si dice che in un paese di media tutti mangiano mezzo pollo al mese, ma c'è chi ne mangia uno e chi zero, la varianza piccola indica l'omogeneità della popolazione. La varianza però ha dentro un quadrato, ad esempio se stiamo misurando l'altezza della popolazione italiana e troviamo la media, la varianza ha un quadrato, quindi dobbiamo usare la deviazione standard (la radice quadrata della varianza) che ha la stessa unità di misura della media.

$$\frac{\sigma_k}{\langle k \rangle} = \left[ \frac{1-p}{p} \frac{1}{(N-1)} \right]^{\frac{1}{2}} \approx \frac{1}{(N-1)^{1/2}}$$

Esprimere media più meno la deviazione standard da subito l'idea di com'è distribuita la popolazione. Supponiamo però di non avere la media ma solo la deviazione standard, ci dice poco, perchè dobbiamo sapere l'unità di misura a cui si riferisce, quindi per capire veramente quanto è concentrata la popolazione intorno alla media bisogna mettere a paragone media e deviazione standard, è il motivo per cui si usa la deviazione standard diviso la media che è un indice di variabilità già riscalato sulla media. E' interessante perchè va come 1 diviso radice di  $N$ , che significa che tanto più la rete è grande tanto più la distribuzione del grado dei nodi è piccata intorno alla media, cioè tutti i nodi hanno grado abbastanza simile.

## Poisson distribution

Se  $n$  è grande e  $k$  è piccolo la distribuzione binomiale viene approssimata con la rete di Poisson, quindi il modello di rete random viene anche chiamato modello di rete poissoniano. E' una distribuzione più facile da trattare e con  $N$  abbastanza grande ha una forma di tipo Gaussiano (una campana intorno alla media).

$$P(k) = e^{-\langle k \rangle} \frac{\langle k \rangle^k}{k!}$$

## Degree Distribution

Ci piace di più perchè ha un unico parametro, che è il valor medio del grado. Vale anche a dire che tolgo la dipendenza dalla grandezza della rete. Il centro

è la media della distribuzione, che per noi è il valor medio del grado, ed ha una deviazione standard che è la radice del valor medio del grado. Abbiamo scoperto che la distribuzione del grado è una distribuzione a campana simile a quella Gaussiana dove la media è il valor medio del grado e la deviazione standard è la radice del valor medio del grado. Senza fare esperimenti sappiamo tutto sulla distribuzione del grado in una rete di herbest coso.

In una rete random le differenze in termini di grado sono tante? in una rete random possono coesistere nodi con grado alto e nodi con grado basso? Facciamo un esempio numerico, c'è una teoria che dice che in media una persona conosce in media 1000 persone, che è il valor medio del grado. La deviazione è la radice del grado medio. Vuol dire che abbiamo una deviazione standard bassa quindi tutti abbiamo un numero di amici molto simile.

Da ogni distribuzione si può calcolare il massimo valore che la proprietà ha secondo la distribuzione. In una rete random ci chiediamo qual'è il massimo grado che un nodo può avere nella rete, troviamo che è 1185 ed il grado minimo è 816 (non chiede i conti). Se la società umana fosse una rete random avremmo che tutti gli individui avrebbero un numero di amici molto simile, tutti intorno ai mille. Proviamo a calcolare secondo la distribuzione di Poisson la probabilità che un individuo abbia un numero di amici maggiore di duemila, troviamo una probabilità infinitesima, questo vuol dire che le reti random non sono in grado di prevedere individui con un numero di connessioni che sono il doppio di quello medio. Questa cosa non è realistica, chiaramente le reti sociali sono molto lontane dall'essere random.

## Reti sociali

Siamo prima del nuovo millennio (1998), non avevamo ancora i dati dal web. Si sono chiesti quindi come estrarre i dati dal web. Allora l'unico modello che si aveva in testa era quello di rete random. L'idea quindi era di prendere i dati dal web e fare vedere come il modello di rete random dimostri che è simile alla rete del web. Decidono di mappare l'indirizzo nd.edu da cui ottiene una mappa di un dominio del web, scopre che esistono nodi con più di 500 link (impossibile secondo il modello random), decide quindi di calcolare la distribuzione del grado (empirica), trova che la probabilità di avere nodi con link maggiori di 100 è  $10^{-94}$ . Plottando la distribuzione del grado su un grafico con scala logaritmica scopre che è totalmente diversa da quella di poisson.

Andando avanti con gli studi si vedrà che questa non è solo una proprietà delle reti sociali.

Viene in mente che questa distribuzione (con una lunga coda) è la distribuzione di Pareto, ha studiato la regola dell'80/20. In generale quello che ha studiato è il fatto che in tutti i sistemi umani c'è una grande disomogeneità tra le persone. Nella distribuzione del nodo nelle reti sociali si ha questa grande disomogeneità (l'80% dei nodi ha l'1% dei link). Oltre alla regola da la distribuzione di Pareto, che se plottata su una scala logaritmica da una retta.

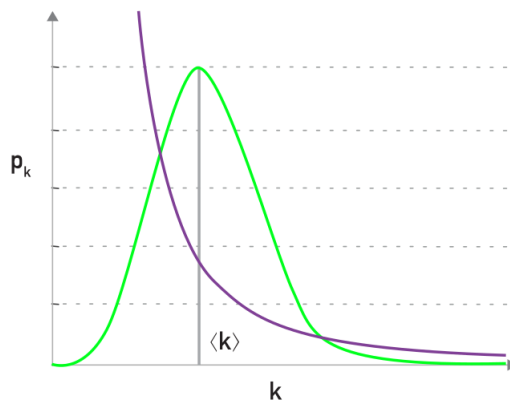
Per valori bassi del grado nelle reti sociali la distribuzione è molto più alta delle reti random, significa che nelle reti random molti nodi hanno lo stesso

grado, mentre nelle reti sociali no. Nelle reti sociali ho un numero elevato di nodi con grado molto piccolo, non previsto dai valori random. Per valori intorno alla media abbiamo che la rete random ha un numero di nodi intorno alla media che è maggiore della rete sociale, perchè una rete random ha molti nodi simili. Per grandi valori del grado la differenza è evidente, perchè in una rete random non si possono avere nodi con un grado molto maggiore della media mentre in reti sociali si vede un numero elevato di nodi con valore del grado molto più elevato di quello della media. Quindi le reti random non sono in grado di prevedere l'esistenza degli influencer (hub), che invece vengono previsti dalle reti sociali.

## Scale-Free Network

Si dice una scale-free network una rete la cui distribuzione del grado è la power law (o heavy tail). In generale non si ottengono distribuzioni del grado che sono power law, ma con forma di heavy tail, non simmetrica (coda lunga sui valori elevati).

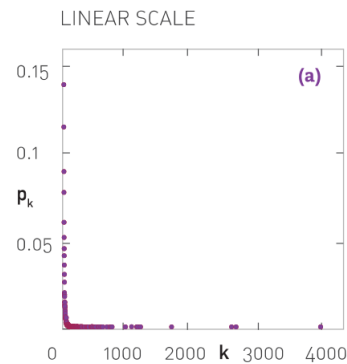
La stessa cosa si vede se si calcola il massimo grado possibile in entrambe le reti. In una distribuzione random si ottiene una distribuzione logaritmica mentre nelle scale free sale esponenzialmente.



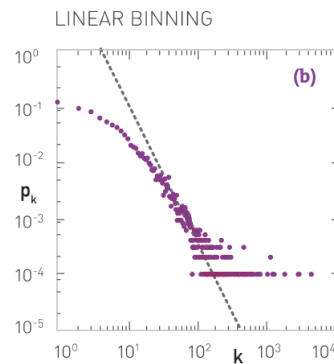
Si chiama scale free network perchè non abbiamo una scala di riferimento per il grado del nodo. L'altra proprietà è chiamata universale, perchè la ritroviamo in tantissimi sistemi diversi, significa che abbiamo trovato una proprietà non dipendente dal sistema che stiamo trovando ma è propria in tutte le reti, significa che tutte le reti hanno ragione di esistere indipendentemente da cosa stiamo modellando. E' qui che parliamo di network science, perchè indipendentemente dal sistema possiamo studiare le reti con un corpus teorico simile. Facendo il fitting delle distribuzioni (ho i dati empirici, costruisco la distribuzione empirica e trovo la distribuzione teorica che meglio approssima la distribuzione empirica) ottengo distribuzioni che stanno tra 2 e 3.

Non tutte le reti sono scale-free, se abbiamo dei vincoli (ad esempio le reti chimiche) hanno una scala.

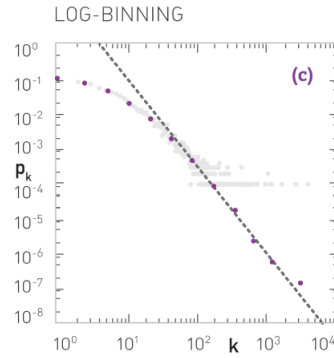
Come si fa il plot della distribuzione del grado, se usiamo una scala lineare otteniamo la distribuzione ma illeggibile:



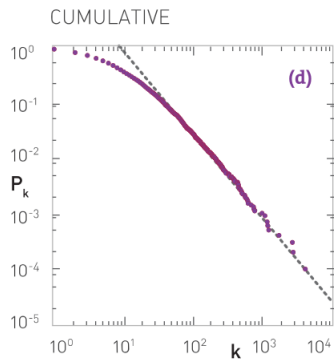
quindi si usa la scala log del grado e log della probabilità. Capita che abbiamo delle righe orizzontali, perchè abbiamo la scala del grado che va da 0 a 10k, ma magari abbiamo tanti nodi vicino a 10k (un nodo con grado diverso), vengono approssimati a 10k:



Un'alternativa è usare il binning logaritmico, quando plottiamo qualcosa invece che calcolare le frequenze di ogni grado calcoliamo le frequenze per ogni intervallo:



Oppure plottiamo la CCDF, la probabilità che il grado sia maggiore di un certo grado fissato, in questo modo otteniamo un bel plot della distribuzione del grado:



Il concetto fondamentale è che c'è qualcosa nelle reti sociali che fa in modo che esistano dei nodi con grado molto maggiore e minore della media, questo discosta le reti sociali dalle reti random. Questo è modellato su modelli heavy tail. Noi misuriamo per ogni nodo il grado, ne facciamo la distribuzione empirica e poi per capire se si sovrappone a quella random sovrapponiamo la distribuzione di poisson. Un altro modo più semplice è calcolare la media e la mediana (il valore centrale), mentre in una distribuzione simmetrica come Poisson la media e la mediana coincidono, in una distribuzione con una lunga coda abbiamo tipicamente una mediana di qualche unità, perchè abbiamo tantissimi nodi con grado molto molto basso, ed una media che è più spostata verso la coda, perchè è la somma di ogni valore del grado per la sua probabilità. Nelle distribuzioni del grado in sistemi non omogenei abbiamo che la mediana è molto minore della media e questo già ci dice che siamo in un sistema power law.



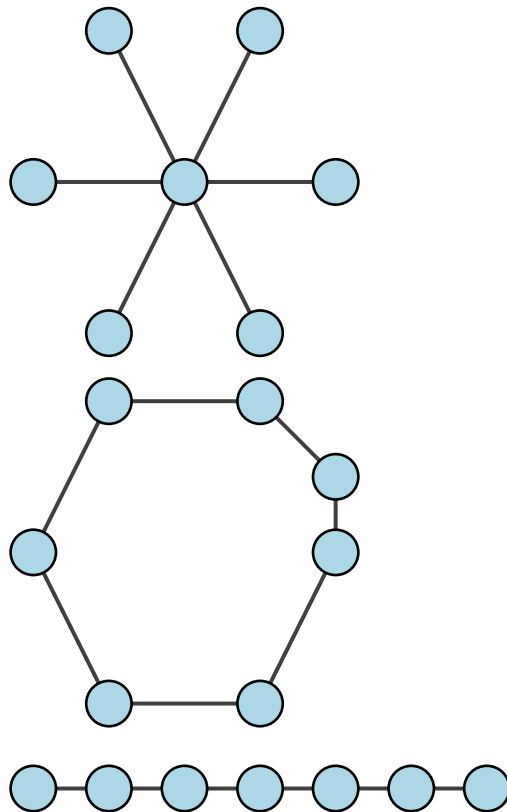
# Misure di centralità

Il concetto di centralità è intuitivamente capibile ma è impossibile darne una definizione unica, perchè ci possono essere varie accezioni di nodo centrale della rete, alcune di queste serve in un caso e non nell'altro. Quello che vogliamo in pratica è fare un ranking dei nodi, per fare questo dobbiamo trovare una metrica su cui fare il ranking. Misure di centralità:

- Definiamo il concetto di centralità
- Troviamo la metrica corrispondente
- Calcoliamo la metrica su tutti i nodi
- Facciamo il ranking dei nodi

Degli studiosi introducono tre piccoli grafi per capire il concetto di centralità di un nodo:

- Un grafo a stella, è chiaro che il nodo centrale è quello centrale
- Un cerchio, è intuitivo capire che tutti i nodi hanno la stessa importanza
- Un grafo lineare, diversi concetti di linearità portano a classifiche differenti



## Degree centrality

In un grafo non orientato possiamo dire che misuriamo la centralità di un nodo in termini di connessioni che un nodo ha, tanti più nodi sono connessi su quel nodo tanto più è alta la centralità. Possiamo quindi definire la centralità secondo il degree di un nodo B il grado del nodo i. (è il grado del nodo, ovvero il numero di link che il nodo ha).

Vediamo con questo concetto come risulta il grado dei nodi dei tre grafi visti prima:

- Nodo centrale ha grado 6, tutti gli altri grado 1. Ci piace, perchè il nodo centrale deve avere un valore per la centralità molto più elevato rispetto a quelli periferici.
- Tutti i nodi hanno grado 2, quindi la degree centrality di tutti i nodi è 2. Siamo contenti, perchè in un grafo circolare tutti i nodi devono avere la stessa centralità
- Tutti i nodi tranne i due estremi hanno grado 2, mentre i due estremi hanno grado 1. Qui non siamo troppo contenti, perchè potremmo volere una centralità che tiene conto del fatto che ci sono nodi più centrali.

Potrei desiderare di comparare la centralità su reti diverse. La centralità in termini di numero di connessioni dipende da come è fatta la rete, quindi difficilmente posso comparare la centralità dei nodi su reti diverse, a meno che non NORMALIZZI. Posso volere i valori assoluti se sto misurando qualcosa che sta sulla stessa rete, ma se volessi paragonarla con un'altra network non potrei paragonare con i valori assoluti. Normalizzare significa togliere da una certa metrica la dipendenza da un parametro, ad esempio togliere la dipendenza di quanti nodi ha una rete (divido tutto per n-1):

$$C_d^{norm}(v_i) = \frac{d_i}{n-1}$$

N-1 è anche il massimo valore possibile del grado dei nodi, sto quindi normalizzando il grado di un nodo rispetto al grado massimo che potrebbe avere. normalizzare per il numero massimo è problematico se abbiamo reti con tanti nodi, perchè otteniamo dei ranking molto piccoli e molto simili. Quindi un altro modo è dividere per il massimo grado che osservo nella rete, con l'idea che tanto più la rete è grande tanto più ci saranno nodi con degree elevato. Otteniamo il vantaggio di dividere per numeri più piccoli:

$$C_d^{max}(v_i) = \frac{d_i}{\max_j d_j}$$

Rimane il problema dato dagli outlier, potrei avere un nodo outlier con grado elevatissimo, la normalizzazione migliore è normalizzare per la somma della metrica che si sta usando, divido per la somma dei gradi dei nodi, che sappiamo essere 2 volte il numero di link.

$$C_d^{max}(v_i) = \frac{d_i}{\sum_j d_j} = \frac{d_i}{2|E|}$$

Qualsiasi normalizzazione deve portare la metrica nell'intervallo 0-1.

Tutti questi modi per dividere la degree centrality sono normalizzazioni, perchè portano i valori in 0-1.

|              | STAR   |        | CIRCLE | LINE   |           |           |           |
|--------------|--------|--------|--------|--------|-----------|-----------|-----------|
|              | Node 1 | Others | All    | Node 1 | Nodes 2,3 | Nodes 4,5 | Nodes 6,7 |
| $C^{degree}$ | 6      | 1      | 2      | 2      | 2         | 2         | 1         |
| $C^{norm}$   | 6/6=1  | 1/6    | 2/6    | 2/6    | 2/6       | 2/6       | 1/6       |
| $C^{max}$    | 6/6=1  | 1/6    | 2/2=1  | 2/2    | 2/2       | 2/2       | 1/2       |
| $C^{sum}$    | 6/12   | 1/12   | 2/14   | 2/12   | 2/12      | 2/12      | 1/12      |

Esercizi: La mediana, il valore del grado tale per cui ho probabilità 0.5 che un nodo abbiamo probabilità minore uguale e 0.5 che abbia probabilità maggiore. Significa che devi cercare 0.5 sull'asse delle y (dai grafici ECDF) e vedere in corrispondenza il valore che abbia valore corrispondente.

La probabilità che un nodo abbiamo grado minore uguale a 2, basta prendere il grado del nodo sull'asse delle x del grafico ECDF e vedere la probabilità sull'asse y. Serve per fare pre processing, scartiamo dei nodi che hanno grado troppo basso o alto.

La probabilità che un nodo abbia un grado maggiore uguale a 7, basta plottare dal grado del nodo sull'ECDF sull'asse y. Uso questo quando voglio trovare gli influencer, voglio capire quando definire secondo il grado quando un nodo è centrale, posso fissare un grado secondo cui definisco sopra quello influencer, ma devo capire qual'è la probabilità che esistano quei nodi.

Per calcolare il numero di nodi, prendo la probabilità e la moltiplico al numero totale di nodi.

Un quantile è quel valore sull'asse delle x (grado) tale per cui la probabilità che un nodo abbia valore minore uguale al grado fissato è il valore del quantile. Quantile 0.1 è il valore del grado per cui la probabilità che il nodo abbia grado minore del quantile è 0.1. (trovo la probabilità indicata dal quantile, sull'asse y, e ritorno il corrispettivo valore sull'asse delle x)

I quartili, sono 0,25; 0,5; 0,75 quantili.

La media non si può calcolare direttamente da questi grafici, dovremmo costruire la densità di probabilità.

Tutto questo serve a filtrare i gradi bassi e a prendere i nodi centrali se prendiamo i gradi alti.

## Reti orientate

Esistono gradi in degree ed out degree. L'in degree è legato al prestigio del nodo, tante più connessioni entrano tanti più followe ho. Quando parliamo di influencer stiamo usando come misura di centralità l'in degree del nodo. Però anche l'out degree di un nodo da un'idea di centralità. Da l'idea che un nodo voglia espandersi nella rete. Immaginiamo di entrare in una rete sociale, per diventare famosi cerchiamo di fare link, quindi tanto più un nodo in una rete ha out degree elevato diciamo che è centrale perchè tenta di espandersi. Se vogliamo vedere in una rete sociale giovane quali nodi vogliono essere centrali possiamo misurare l'out degree. Chiaramente potrei anche sommare in od out ma non lo fa mai nessuno.

Misuriamo la centralità con l'idea che tanto è più centrale un nodo tante più connessioni ha. Ma dicevamo che nella rete lineare non siamo tanto contenti della misura, perchè quelli centrali non risaltano. C'è un concetto di centralità che risponde a questo quesito?

## Betweenness centrality

E' definita sulle reti orientate e sulle non, ma si usa sulle reti orientate. Un concetto di centralità diverso dal numero di connessioni, può essere che un nodo stia sui cammini di informazione della rete. Pensiamo ad esempio al gossip, potrei avere meno connessioni ma essere su tutti i flussi di gossip, c'è un concetto di centralità che un nodo potrebbe avere meno connessioni ma più visibilità sulle informazioni che girano sulla rete? Ad esempio i router su cui passano più pacchetti sono più importanti di quelli che hanno tante connessioni, ancora di più quando si valutano le reti aziendali, è vero che un nodo con tante connessioni è centrale, ma possono esserci nodi con non tante connessioni ma che sono a conoscenza di tante cose. Ancora di più nelle reti di energia, ho un nodo con poche connessioni ma tutta l'energia passa da quel nodo.

Dobbiamo fare due assunzioni:

- Tutte le coppie di nodi scambiano la stessa quantità di informazione. Vogliamo una metrica che non tiene conto del numero di informazioni che ci scambiamo.
- Tutte le informazioni viaggiano sui cammini a distanza minima

Diciamo che la Betweenness Centrality di un nodo  $v$ :

$$C_b(V_i) = \sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

considero tutte le coppie di nodi  $s$  e  $t$  della rete diverse da  $v$ . Per ogni coppia di nodi calcolo questo rapporto, il numeratore è il numero di cammini minimi da  $s$  a  $t$  e viceversa, che passano da  $v_i$ .

Al numeratore ho il numero totale di cammini minimi da  $s$  a  $t$ .

Possiamo interpretare questa cosa come una specie di probabilità che l'informazione che passa da  $s$  a  $t$  passi da  $v$ .

Poi sommo su tutte le coppie  $s$  e  $t$ .

Da un'informazione su quali cammini minimi di informazione vedono  $v_i$  come nodo interno.

Attenzione che quando faccio il conto, se anche sono in un grafo non orientato, il cammino  $(s, v_i, t) \neq (t, v_i, s)$ .

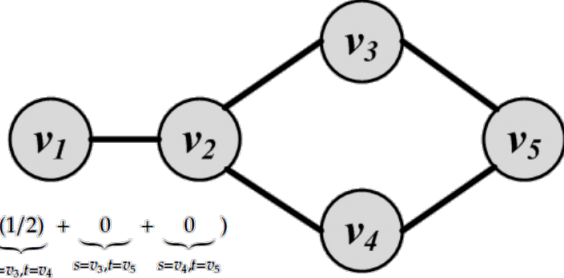
Ovviamente sto sommando frazioni, posso quindi avere una metrica maggiore di uno, dato che ci piacciono metriche tra 0 e 1 normalizzo, uso il valore massimo della Betweenness centrality, ovvero quando  $\frac{\sigma_{st}(v_i)}{\sigma_{st}} = 1$ .

$$C_b(v_i) = \sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}} = \sum_{s \neq t \neq v_i} 1 = 2 \binom{n-1}{2} = (n-1)(n-2)$$

Notiamo che il valore massimo è una distribuzione binomiale, quindi scriviamo la Betweenness Centrality Normalizzata:

$$C_b^{norm}(v_i) = \frac{C_b(v_i)}{2 \binom{n-1}{2}}$$

Per fare gli esercizi, per il nodo prendo tutte le coppie di nodi che non comprendono anche il nodo in studio, calcolo la frazione tra il cammino minimo che passa dal nodo e il numero di cammini minimi totali tra i due nodi (anche quelli che non passano dal nodo che sto studiando). Sommo tutti i contributi e trovo la Betweenness centrality del nodo.



$$C_b(v_2) = 2 \times \left( \underbrace{(1/1)}_{s=v_1, t=v_3} + \underbrace{(1/1)}_{s=v_1, t=v_4} + \underbrace{(2/2)}_{s=v_1, t=v_5} + \underbrace{(1/2)}_{s=v_3, t=v_4} + \underbrace{0}_{s=v_3, t=v_5} + \underbrace{0}_{s=v_4, t=v_5} \right)$$

$$= 2 \times 3.5 = 7,$$

$$C_b(v_3) = 2 \times \left( \underbrace{0}_{s=v_1, t=v_2} + \underbrace{0}_{s=v_1, t=v_4} + \underbrace{(1/2)}_{s=v_1, t=v_5} + \underbrace{0}_{s=v_2, t=v_4} + \underbrace{(1/2)}_{s=v_2, t=v_5} + \underbrace{0}_{s=v_4, t=v_5} \right)$$

$$= 2 \times 1.0 = 2,$$

$$C_b(v_4) = C_b(v_3) = 2 \times 1.0 = 2,$$

$$C_b(v_5) = 2 \times \left( \underbrace{0}_{s=v_1, t=v_2} + \underbrace{0}_{s=v_1, t=v_3} + \underbrace{0}_{s=v_1, t=v_4} + \underbrace{0}_{s=v_2, t=v_3} + \underbrace{0}_{s=v_2, t=v_4} + \underbrace{(1/2)}_{s=v_3, t=v_4} \right)$$

$$= 2 \times 0.5 = 1,$$

## Closeness Centrality

La closeness centrality è una misura che sta tra le Betweenness e la degree. Quando diciamo che misuriamo la centralità di un nodo in base al numero di link intendiamo che un nodo è in grado di far arrivare la conoscenza a molti nodi della rete, ma la Betweenness centrality è ad 1 hop, ora cerchiamo di allargarci. L'idea è che tanto più un nodo è centrale tanto più riesce a far arrivare l'informazione in modo efficiente agli altri nodi della rete. Quando diciamo in modo efficiente dobbiamo tradurlo in ottica di reti, traduciamo questo in cammino minimo.

Le informazioni in una rete passano tra i link di nodo in nodo, diciamo che un nodo è tanto più centrale tanto più rapidamente riesce a propagare l'informazione nella rete. Quindi se un nodo ha in media cammini minimi di lunghezza bassa allora in pochi hop raggiunge gli altri nodi.

Quindi la lunghezza dei cammini minimi tra il nodo che consideriamo centrale agli altri ci dà la centralità di quel nodo.

Quindi definiamo la closeness centrality di un nodo iniziando a calcolare il cammino minimo di un nodo a tutti gli altri nodi della rete, calcoliamo la media di tutti i cammini minimi, che sono  $n-1$ :

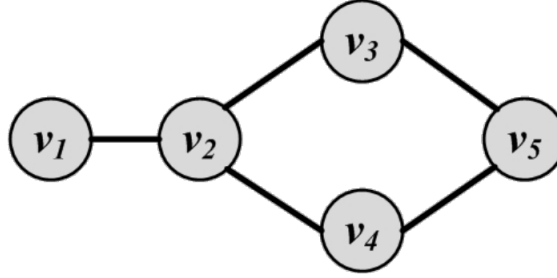
$$\bar{l}_{v_i} = \frac{1}{n-1} \sum_{v_j \neq v_i} l_{i,j}$$

Questa definizione ha un problema, valori bassi della lunghezza media dei cammini corrispondono a nodi più centrali, quindi avremmo una definizione di centralità in modo inverso a quello intuitivo. Questa cosa non ci piace, quindi invece che considerare la lunghezza media dei cammini minimi tra un nodo e tutti gli altri consideriamo il valore inverso:

$$C_c(v_i) = \frac{1}{\bar{l}_{v_i}}$$

Così ci piace di più perché tanto più è alta tanto più è centrale il nodo.

Notiamo che è simile alla Betweenness però dà comunque un valore anche ai nodi periferici.



$$C_c(v_1) = 1/((1 + 2 + 2 + 3)/4) = 0.5,$$

$$C_c(v_2) = 1/((1 + 1 + 1 + 2)/4) = 0.8,$$

$$C_c(v_3) = C_b(v_4) = 1/((1 + 1 + 2 + 2)/4) = 0.66,$$

$$C_c(v_5) = 1/((1 + 1 + 2 + 3)/4) = 0.57$$

## Eigenvector Centrality

Magari con la degree centrality trovo che un nodo ha molte connessioni con nodi ininfluenti nella rete e trovo un altro che ha meno connessioni ma con nodi più centrali nella rete. Cosa è meglio avere? In alcuni social network (Linkedin) preferiscono questa misura. Si è tentato di introdurre una misura di centralità in cui si nota che non tutte le connessioni hanno lo stesso peso. E' una generalizzazione della degree centrality che pesa le connessioni in modo non omogeneo, significa che da un peso diverso alla connessione in dipendenza a quanto importante è il nodo a cui ci si connette.

Ha un problema, questa misura dipende dal numero di connessioni e dalle Eigenvector Centrality che a loro volta devono calcolarla dalla Eigenvector centrality, è una funzione ricorsiva. Ma è un ricorsività facilmente risolvibile. Sommo su tutti i vicini l'Eigenvector Centrality dei vicini, so che sono vicini perchè ci metto una matrice di adiacenza.

Riconosciamo che è un'equazione di autovettori ed autovalori, che dal punto di vista matematico possiamo risolverla automaticamente, significa che il vettore dell'Eigenvector Centrality è un autovettore della matrice di adiacenza e  $\lambda$  è il corrispondente autovalore.

In pratica data la matrice di adiacenza della rete ne calcoliamo tutti gli autovalori, prendiamo l'autovalore più alto, calcoliamo l'autovettore corrispondente e abbiamo trovato il vettore dell'Eigenvector Centrality.

## Reti orientate

Nelle reti orientate si ottengono due autovettori, quello di destra è legato agli in-link e quello di sinistra è legato agli out-link. Quando ho una matrice di adiacenza

per una rete orientata (non simmetrica) ottengo due insiemi di autovettori (destrò e sinistro), quello destrò tiene conto dell'in degree e quello sinistro dell'out degree.

## Katz Centrality

Quando facciamo l'Eigenvector Centrality sui grafi orientati nasce un problema. Vediamolo sull'esempio, il nodo 1 non ha link entranti e Supponiamo di voler calcolare l'in Eigenvector Centrality, allora chiaramente l'in Eigenvector Centrality del nodo 1 è 0. Il nodo 2 ha un solo link entrante, quindi quando calcola la sua Eigenvector Centrality eredita l'Eigenvector Centrality del nodo 1, quindi anche la sua sarà 0. E così via si propaga. Questo non ci piace molto, perchè ad esempio il nodo 2 ha almeno 1 link entrante e così ci viene tutto 0. Per risolvere questo problema è stata introdotta la Katz Centrality, che aggiunge una costante  $\beta$  quindi anche se un nodo eredita solo Eigenvector Centrality = 0 dai suoi vicini, lui non avrà comunque un valore nullo. Faccio in modo che anche il numero di connessioni abbia un valore e introduce anche un parametro  $\alpha$  per bilanciare a nostro piacimento la parte di ereditarietà e quanto è importante solo il termine costante di ogni connessione. Tanto più  $\alpha$  è grande, vuol dire che è più importante l'Eigenvector Centrality ereditata dai vicini, se fisso  $\beta$  grande do più importanza alla connessione. (Entrambi i termini sono fissati da noi come vogliamo). Di solito si seleziona  $\alpha$  che sia minore di 1 fratto l'autovalore massimo.

## Page Rank

Consideriamo il problema inverso, supponiamo di avere un nodo che è un hub (ha elevatissima centralità). Quando siamo, ad esempio, nella Katz Centrality se abbiamo un nodo nella rete che ha centralità elevatissima e per caso un nodo che non ha nessuna importanza nella rete ha una connessione verso questa autorità, immediatamente questo nodo eredita tutta la centralità dell'autorità. Questa cosa intuitivamente capiamo non andare bene, perchè così anche nodi non influenti possono diventare importantissimi. Quindi per questo motivo è stato introdotto un nuovo algoritmo, il Page Rank. Che è stato il primo algoritmo di Google. E' simile all'Eigenvector Centrality, ma divide l'Eigenvector Centrality dei nodi vicini per i link uscenti da quel nodo.



# Connettività

Attraversare un grado significa passare da un nodo attraverso un link andando ad un altro nodo.

La walk è una sequenza di link incidenti che visito uno dietro l'altro. Un path è una sequenza di nodi e link dove i nodi sono distinti, la lunghezza di un cammino è il numero di link visitati lungo il cammino. Fissato che parleremo solo di cammini, si dice che un grafo è connesso se esiste un cammino tra ogni coppia di nodi.

## Componenti connesse

Le componenti connesse sono le isole che formano il grafo, la definizione:

1. Una componente connessa è un sottografo della rete tale per cui esiste un cammino da ogni membro del sottografo ad ogni altro membro del sottografo
2. e non esiste nessun altro nodo della rete che può essere aggiunto alla componente connessa preservando questa proprietà.

La definizione data non è abbastanza dettagliata, perchè cosa vuol dire prendere un sottografo? si intende un sottografo generato da nodi, un sottoinsieme di nodi del grafo e tutti i link tra di loro. La seconda proprietà dice che non esiste un cammino tra un nodo della componente connessa e ciascun altro nodo che non sta nella componente connessa.

## Componenti connesse nei social network

Nelle reti sociali è che si ha un enorme componente connessa centrale che comprende la maggior parte (sopra il 90%) dei nodi. Questo avviene anche in reti economiche, altre reti invece sono grafi connessi, ad esempio Internet.

E' interessante capire quanto sono grandi le componenti connesse, se ho una grande componente che include la maggior parte dei nodi, avremo (in scala logaritmica) una componente connessa comprendente la maggior parte dei nodi della rete, tutte le altre componenti connesse hanno dimensione molto piccola.

Possiamo iniziare a ragionare se può esistere una rete sociale divisa in due componenti connesse distinte ed uguali. Campiamo che è impossibile avere una divisione del genere, per il puro calcolo combinatorio. Se ci aggiungiamo che in una rete sociale tipicamente chi si iscrive vuole poter comunicare con il resto della rete è improbabile avere due isole uguali.

## Componenti connesse in reti orientate

Un grafo orientato è fortemente connesso se esiste un path tra tutte le coppie di nodi. In un grafo orientato, una componente connessa è fortemente connessa se è un massimo sottoinsieme di nodi tale per cui ogni nodo può raggiungere ed essere raggiunto da tutti gli altri nodi della componente.

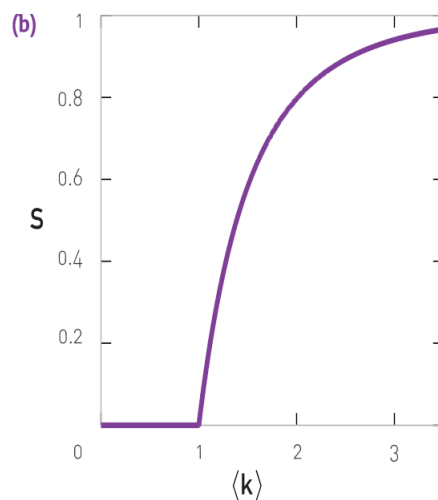
## Largest connected component and Giant component

La più grande componente connessa in un grafo è la componente connessa che contiene il più grande numero di nodi. La Giant Component è una componente connessa, la cui dimensione (numero di nodi) cresce in proporzione al numero di nodi  $N$ .

### Random Network

#### Growing a random network

Ci aspettiamo che al crescere del grado medio la rete diventi più connessa tanto da avere una grossa componente connessa, ma abbiamo visto che c'è una transizione di fase, c'è un punto in cui cambia drasticamente la struttura della rete.



Cominciando con  $N$  nodi isolati, i link sono aggiunti gradualmente con un processo random. Questo corrisponde a un aumento graduale di  $p$ , con conseguenze sulla topologia della rete. Per quantificare questo processo, esaminiamo innanzitutto come varia la dimensione del più grande cluster più grande della rete,  $N_G$ , varia con il grado medio  $\langle k \rangle$ .

$N_G$  è il numero di nodi nel giant component.

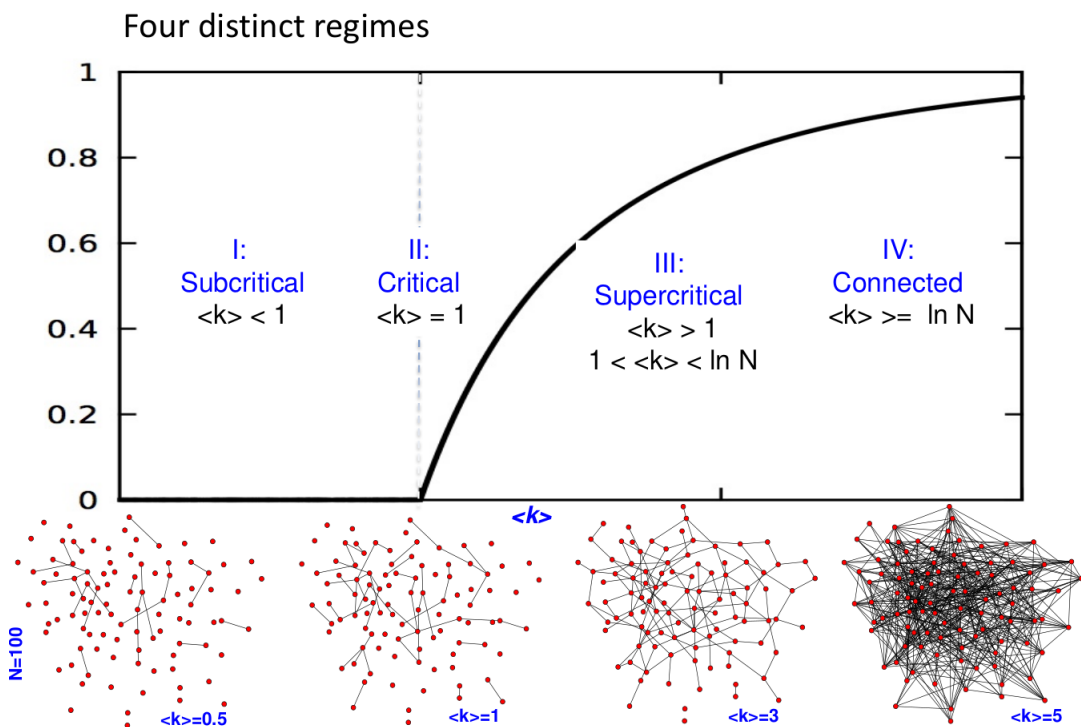
$S = N_G/N$   $S$  è la frazione di nodi nella componente più grande componente connessa. Ci si aspetterebbe che la componente più grande componente più grande cresca gradualmente. Tuttavia, questo non è il caso.  $N_G/N$  rimane zero per piccoli  $\langle k \rangle$ , il che indica l'assenza di una grande componente connessa. Quando  $\langle k \rangle$  supera un valore critico (1),  $N_G/N$  aumenta, segnalando il rapido emergere di una componente grande componente che possiamo chiamare componente gigante. Erdős e Renyi nel loro classico

articolo del 1959 prevedevano che la condizione per l'emergere di una grande componente connessa componente connessa è:  $\langle k \rangle = 1$ .

E' strano notare che se abbiamo un valor medio del grado a 2 si ha una gigante componente connessa.

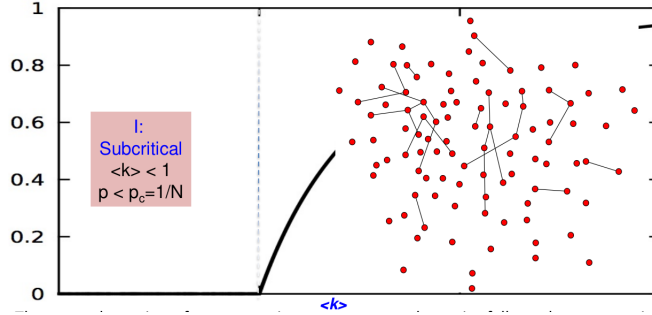
In altre parole, abbiamo una componente (potrebbe essere un gigante) se e solo se ogni nodo ha in media più di un collegamento. Il fatto che abbiamo bisogno di almeno un collegamento per nodo per osservare una componente grande non è inaspettato. Infatti, perché esista una grande per esistere, ogni suo nodo deve essere nodi deve essere collegato ad almeno un altro nodo. altro nodo. È in qualche modo controintuitivo, tuttavia, che un solo collegamento sia sufficiente.

Vediamo che in realtà si distinguono 4 regione, nel modello di reny, per come si formano le giant connected component nelle reti random. Quando il valor medio del grado è uguale a 1 ho il cambiamento di fase, in realtà in mezzo ci sono alcune regioni.



Si chiama regione sub critica quando il valor medio del grado è minore di 1. Si ha una regione critica quando si ha il valore di soglia. Una regione super critica, in cui il valore medio del grado sta tra 1 ed il  $\ln N$ . Ed una regione connessa in cui il valore del grado è maggiore del  $\ln N$ .

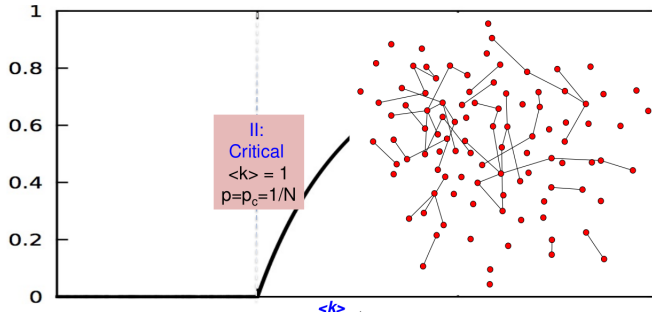
## La regione sotto critica



Il valor medio del grado è minore di 1. Il rapporto tra il valor medio del grado e  $N - 1$  è il parametro  $p$  della rete di Renyi è  $p = \frac{\langle k \rangle}{N-1}$ . La rete fatta così non è una rete, la maggior parte dei nodi è isolata, o ci sono pochissime componenti connesse. Quindi non solo non possiamo dire che c'è una componente gigante, ma non c'è neanche una grande componente connessa principale. Per calcolare se siamo nel regime di componente giant calcoliamo il numero di nodi nella componente giant fratto il numero di nodi nella rete, se tende a 0 non ho una componente gigante altrimenti sì.

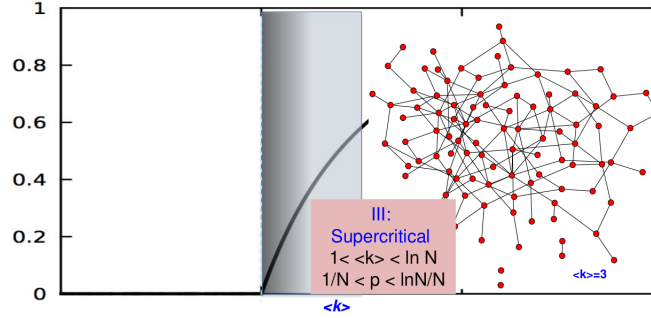
$$\frac{N_G}{N} \approx \ln N$$

## Regione critica



Si ha un valore medio del grado del nodo pari a 1. Ed effettivamente cominciamo ad avere una grossa regione connessa. Però se calcolo  $\frac{N_G}{N} = N^{\frac{1}{3}}$  quindi ancora non si ha una componente gigante, si hanno varie componenti connesse che sono alberi. Se nella nostra società ognuno avesse in media un amico avremmo una componente connessa di 3 milioni. Questo non toglie che non possiamo ancora parlare di una struttura sociale vera e propria.

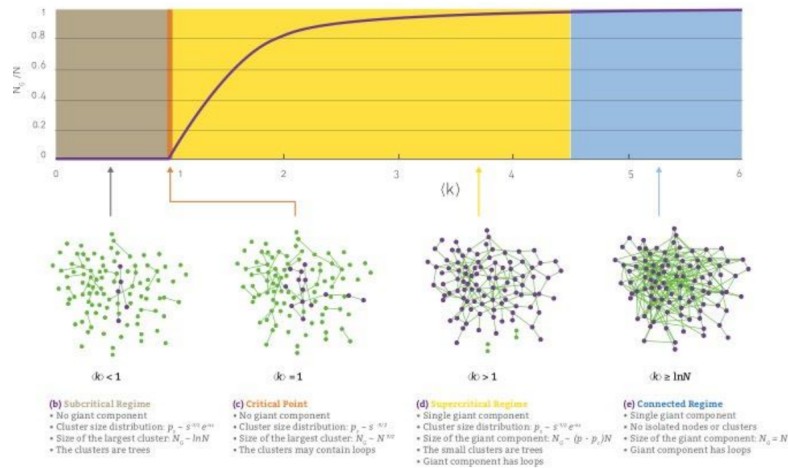
## Regione super critica



Il valor medio del grado sta tra 1 ed il  $\ln N$ , ricordiamoci che il logaritmo è una funzione che abbassa molto, in media in nodi hanno qualche decina di link nel caso della società umana. In questo caso si dimostra che il numero di nodi nella componente connessa grande è  $(p-p_c)$  questo vuol dire che  $\frac{N_G}{N} \approx \langle k \rangle - 1$ , quindi abbiamo trovato una componente connessa giant, perche questo rapporto non tende a zero. Qua abbiamo una vera struttura di rete.

## Regione connessa

Quando il valor medio del grado è maggiore al numero di nodi ottengo un grafo connesso, un unica componente connessa che ovviamente è una giant. Deduciamo che in una rete random basta che i nodi abbiano il valor medio del grado che supera di un po' 1 per entrare in una regione super critica che forma una componente connessa giant. Quando diciamo che quando le reti sociali hanno una componente connessa giant che comprende la maggior parte dei nodi è per semplice calcolo di probabilità, quindi non c'è meccanismo intelligente sul fatto che tutti i nodi stanno insieme.



Dal punto di vista teorico sottolineiamo la differenza che c'è tra una componente connessa grossa ed una componente giant, abbiamo una componente giant quando il rapporto tra il numero di nodi della componente giant e il numero di nodi della rete all'aumentare del numero di nodi rimane costante e non tende a zero.

## Esempi

Vediamo in alcuni reti se le componenti connesse sono legate a quello che abbiamo visto per le reti random.

- Internet: viene fuori che sta nella regione super critica
- Collaborazioni scientifiche: regione super critica
- Rete degli attori: connessione completa, ma non la troviamo nella rete reale, perchè esistono dei film che prevede degli attori che è avulso da tutto il sistema

Quello che è interessante è che le previsioni del modello di Renyi rispetto al modello reali sono molto simili.

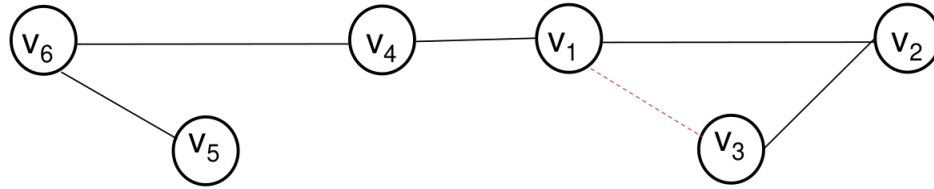


# Transitività

E' una misura mesoscopica, sono misure che coinvolgono un numero limitato di nodi e link. La transitività si misura tramite il coefficiente di clustering, che valgono solo per le reti non orientate.

La transitività è un concetto matematico che conosciamo bene, si dice che una relazione è transitiva se A è in relazione con B e B è in relazione con C allora A è in relazione con C. Vediamo cosa vuol dire in una rete. La relazione di cui si studia la transitività in una rete è la relazione di "connessi da un link" e la transitività nella rete diventa, se il nodo v1 e v2 sono connessi e v2 e v3 sono connessi allora anche v1 e v3 sono connessi, connessi da 1 link. Se vogliamo dirlo in modo social significa che un amico di un mio amico è mio amico. Questa è una nozione bella matematica ma poco applicabile nelle reti sociali reali, perchè non si ha sempre una perfetta transitività. E' una nozione troppo forte per essere usata in una rete sociale. Abbiamo la transitività in senso matematico quando abbiamo un grafo completo, ma basta che 1 link non sia presente per non avere più la perfetta transitività. Quindi il concetto di transitività si ha solo in sottografi completi, chiamati CLIQUE.

Dobbiamo introdurre una transitività parziale. Quello che vogliamo dire è che l'amico del mio amico più probabilmente sarà mio amico rispetto ad un nodo scelto a caso nella rete. Nell'esempio se noi siamo v1 possiamo dire che abbiamo più probabilità di avere come amico v3 rispetto a v5 perchè con v3 abbiamo un amico in comune.



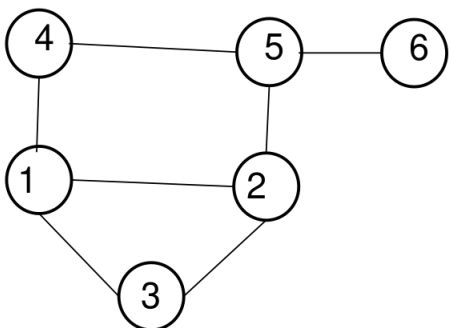
## Coeficiente di Clustering Globale

Si chiama coefficiente di clustering una misura della transitività della rete (non c'entra nulla con il machine learning). Iniziamo dal coefficiente di clustering globale è il numero di cammini di lunghezza 2 che hanno il terzo link diviso il numero di cammini che hanno lunghezza 2. Ci chiediamo qual'è la frazione di cammini che hanno due link che hanno il terzo link che li chiude.

$$\frac{|\text{Paths of Length 2 that have the third edge}|}{|\text{Paths of Length 2}|}$$

Attenzione che quando devo elencare tutti i cammini di lunghezza due devo elencarli in entrambe le direzioni, dato che siamo in un grafo non orientato. Dopo aver elencato tutti i cammini con due link per ognuno mi chiedo se è chiuso da un terzo link o no. Posso anche contare tutto una volta, è come dividere per due il numeratore e denominatore. Lei per fare gli esercizi prende

tutte le coppie il cui nodo centrale sale secondo il valore (prima il nodo centrale è 1...)



$C=6/22=3/11$

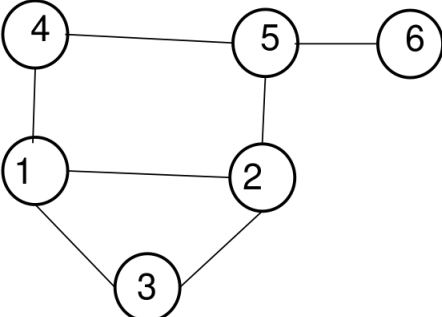
| Path of length 2 | Third edge |
|------------------|------------|
| 213 312          | yes        |
| 214 412          | no         |
| 314 413          | no         |
| 123 321          | yes        |
| 125 521          | no         |
| 325 523          | no         |
| 132 231          | yes        |
| 145 541          | no         |
| 254 452          | no         |
| 256 652          | no         |
| 456 654          | no         |

Triplette

L'idea è che invece di chiedersi se l'amico del mio amico è anche mio amico, ci chiediamo se una coppia di miei amici sono anche amici tra di loro, quindi vedo il coefficiente di clustering globale come la frazione di coppie di nodi che hanno un amico in comune. E' la stessa cosa ma vista in modo diverso.

Possiamo definire il coefficiente di clustering globale tramite le triplette dove diciamo che una tripletta è connessa se ha 3 nodi v1, v2, v3 che sono connessi dai rispettivi link. E' come prima ma non considero i cammini, solo che i nodi nella tripletta siano distinti.

$$\frac{|\text{number of closed triplets}|}{|\text{total number of triplets}|}$$

|  | Triplets | Closed? |
|-----------------------------------------------------------------------------------|----------|---------|
|                                                                                   | 213      | yes     |
|                                                                                   | 214      | no      |
|                                                                                   | 314      | no      |
|                                                                                   | 123      | yes     |
|                                                                                   | 125      | no      |
|                                                                                   | 325      | no      |
|                                                                                   | 132      | yes     |
|                                                                                   | 145      | no      |
|                                                                                   | 254      | no      |
|                                                                                   | 256      | no      |
|                                                                                   | 456      | no      |
|                                                                                   |          |         |

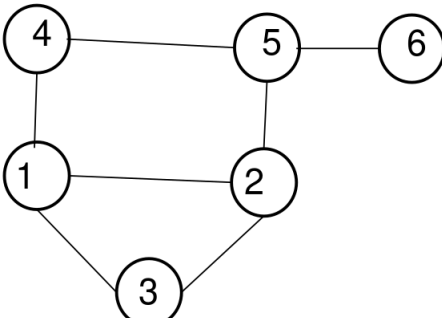
$C=3/11$

### Triangoli

Dico che il numero di triangoli per 6 diviso il numero di cammini di lunghezza due o il numero di triangoli per 3 diviso il numero di triplette.

$$\frac{6 \cdot \text{number of triangles}}{\text{number of paths of length 2}}$$

$$\frac{3 \cdot \text{number of triangles}}{\text{number of triplets}}$$

|  | Path of length 2 | Triangles |
|-------------------------------------------------------------------------------------|------------------|-----------|
|                                                                                     | 213 312          | 123       |
|                                                                                     | 214 412          |           |
|                                                                                     | 314 413          |           |
|                                                                                     | 123 321          |           |
|                                                                                     | 125 521          |           |
|                                                                                     | 325 523          |           |
|                                                                                     | 132 231          |           |
|                                                                                     | 145 541          |           |
|                                                                                     | 254 452          |           |
|                                                                                     | 256 652          |           |
|                                                                                     | 456 654          |           |
|                                                                                     |                  |           |

$C=1 \cdot 6/22=3/11$

## Coeficiente di Clustering Locale

I miei amici tra di loro sono amici? quindi la mia rete è coesa oppure no. Questo equivale a chiedersi qual'è il coefficiente di clustering del singolo nodo. Definiamo coefficiente di clustering del nodo  $v_i$  il numero di coppie di amici del nodo al denominatore e sopra le coppie di amici del nodo connessi da un link. E' la frazione di coppie di miei amici che sono amici tra di loro. Anche in questo caso sta tra 0 e 1.

$$C(v_i) = \frac{\text{number of pairs of neighbors of } v_i \text{ that are connected}}{\text{number of pairs of neighbors of } v_i}$$

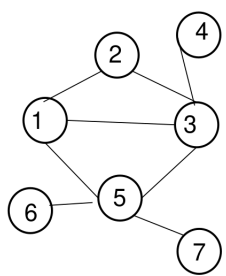
In un grafo non orientato, il denominatore può essere scritto come:

$$\binom{d_i}{2} = d_i \frac{(d_i - 1)}{2}$$

Il denominatore è facile da calcolare, se il nodo ha grado  $d_i$  ho  $d_i$  modi per scegliere il primo amico e  $d_i$  modi - 1 per scegliere il secondo amico il tutto diviso due perchè non c'è ordinamento. Il numeratore è prendere l'elenco di coppie di amici e capire quali sono amici tra di loro.

Questa è una misura di quanto un nodo riesce a creare una rete compatta intorno a se. Notiamo che se un nodo è foglia ha coefficiente di clustering non definito.

Per rendere globale questa misura dobbiamo calcolarla per ogni nodo sulla rete e poi fare la distribuzione, ovvero con che frequenza osservo il coefficiente sulla rete.

| Exercise                                                                            |  | Pairs of neighbours | Connected? |
|-------------------------------------------------------------------------------------|--|---------------------|------------|
|  |  | 213                 | yes        |
|                                                                                     |  | 215                 | no         |
|                                                                                     |  | 315                 | yes        |
|                                                                                     |  | 123                 | yes        |
|                                                                                     |  | 132                 | yes        |
|                                                                                     |  | 134                 | no         |
|                                                                                     |  | 135                 | yes        |
|                                                                                     |  | 234                 | no         |
|                                                                                     |  | 235                 | no         |
|                                                                                     |  | 435                 | no         |
|                                                                                     |  | 153                 | yes        |
|                                                                                     |  | 156                 | no         |
|                                                                                     |  | 157                 | no         |
|                                                                                     |  | 356                 | no         |
|                                                                                     |  | 357                 | no         |
|                                                                                     |  | 657                 | no         |
|                                                                                     |  |                     |            |

$$c(1)=2/3$$

$$c(2)=1/1$$

$$c(3)=2/6$$

$$c(5)=1/6$$

## Rete Erdos-Renyi

Nelle rete sociali il coefficiente di clustering è tendenzialmente alto, perchè vogliamo che i nostri amici siano amici tra di loro. Avviene anche nelle reti random?

In una rete random ogni link ha la stessa probabilità di esistere ed è  $p$ , quindi se il coefficiente di clustering è la probabilità che esista un link tra due amici in comune è sempre  $p$ .

In una rete random il coefficiente di clustering locale dipende dal grado del nodo? in una rete random ogni link ha la stessa probabilità di esistere, quindi è indipendente dal grado, è sempre  $p$  il coefficiente di clustering.

$$c = p = \frac{\langle k \rangle}{N}$$

In una rete sociale invece? in una rete sociale tendenzialmente più grande è il grado del nodo tanto è più piccolo il coefficiente di clustering, perchè il nodo sta creando una rete composta non di amici stretti.

Se plottiamo il coefficiente di clustering medio sul numero di nodi vediamo che il coefficiente di clustering è inversamente proporzionale al numero di nodi, mentre in una rete sociale non così.

Perchè gli amici diventano amici?

- Più opportunità di incontrare amici degli amici
- Trustability, se abbiamo un amico in comune mi fido perchè c'è un mio amico in mezzo
- Incentivo, c'è una forma di incentivo psicologico, il fatto che una persona ha amici che non vogliono essere amici tra di loro è fonte di stress.

Per ogni valore del grado considero tutti i nodi che hanno quel grado e su quel set di nodi calcolo la media dei loro coefficienti di clustering locale (figura 4 del paper di facebook) in realtà dopo che calcolo la distribuzione locale per ogni nodo potrei plottare sul grafico non solo la media ma ad esempio la mediana o i percentili etc.

Diadi

Partiamo da un articolo scientifico, il Granovetter's paper, (anni '50) che studia come la società può essere modellata su una rete. In questo articolo viene scritto che una debolezza fondamentale delle teorie sociologiche correnti è che non riesce a collegare le interazioni a livello microscopico con quelle a livello macroscopico. Gli studi a larga scala riescono a dare una visione significativa dei macrofenomini, la sociologia riusciva a descrivere le grandi strutture sociali ed a livello microscopico si hanno teorie e dati che riescono a distinguere bene i gruppi piccoli, ma non si capisce come le interazioni all'interno dei piccoli gruppi si aggregano per formare le caratteristiche su larghe scale della sociologia. Quindi quello che vogliono fare è trovare il ponte tra le strutture microscopiche e quelle macroscopiche.

## The strenght of ties

L'intuizione è che tutto sta in una caratteristica dei link, la forza. Vede che non tutti i link hanno lo stesso valore, quindi non è una struttura piatta in cui una volta che so il grado dei nodi so tutto, ma devo ragionare sul valore che ha ogni link, il valore dato dalla forza del link. Dove intuitivamente capiamo cos'è la forza di un legame, ma poi dovremo darne una definizione. La forza di un legame è una combinazione di quanto tempo viene speso su questa relazione, l'intensità delle emozioni coinvolte nella relazione, dalla confidenza dei due nodi della relazione ed i servizi reciproci (quanto i due nodi si aiutano). Ovviamente queste caratteristiche sono indipendenti a priori. Quello che dobbiamo capire, è come misurare la forza di un legame in una società.

In generale quello che interessa per capire com'è il ponte è sapere se un legame è forte o debole, da qua viene la suddivisione delle reti sociali in legami forti e deboli. Quindi siamo arrivati a dire che all'interno di una società ci sono legami forti e deboli. I legami forti sono la famiglia, gli amici . . . i legami deboli sono i legami che non ci coinvolgono più di tanto dal punto di vista del capitale sociale. Per capire la struttura di una rete sono importanti i legami forti. Per capire se era vera la dicotomia tra legami forti e deboli effettuano un esperimento: non avendo a disposizione dati online fanno le interviste alle persone, intervista le persone che hanno iniziato un nuovo lavoro a queste persone chiede come hanno scoperto dell'opportunità di lavoro, pensa che siano stati gli amici a dirlo, ma in realtà scopre che la notizia dell'opportunità di lavoro l'ha data una persona con legame debole. Si accorge quindi che le informazioni rilevanti all'interno di una rete sociale sono derivanti dai legami deboli. E' ovvio, dobbiamo trovare una notizia che non conosciamo lontano da me. Quindi quando considero una diade (due nodi collegati) tanto è più forte il legame tra di loro tanto più condividono il capitale sociale intorno a loro, condividono la stessa cerchia sociale, quindi non riescono ad avere informazioni nuove. Vuol dire che in realtà i legami deboli sono altrettanto importanti come i legami forti, soprattutto quando si ha un passaggio di informazioni nella rete, perchè sono quelli che permettono il passaggio di informazioni in parti diverse della rete.

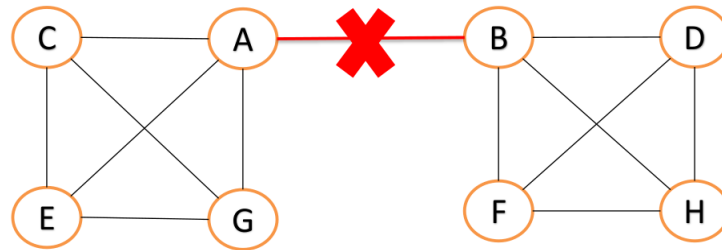
E' capendo questa caratteristica che ha capito il ponte tra le piccole strutture

e la rete. Le isole molto forti di nodi si connettono tramite i legami deboli.

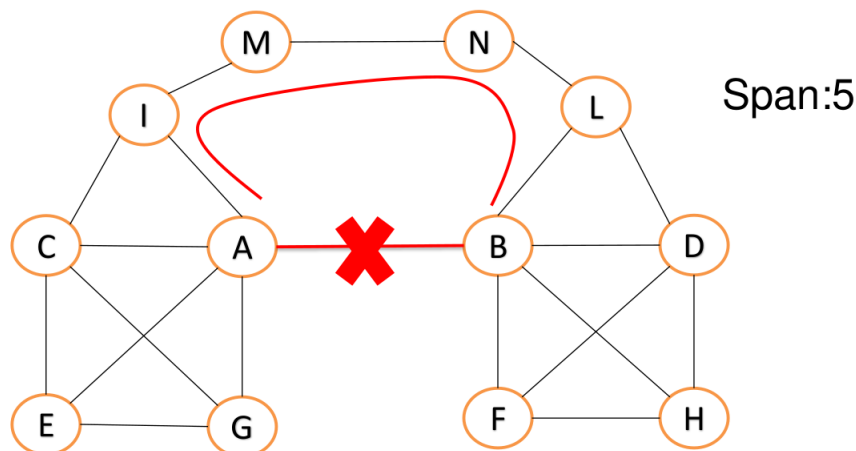
## Bridge tra locale e globale

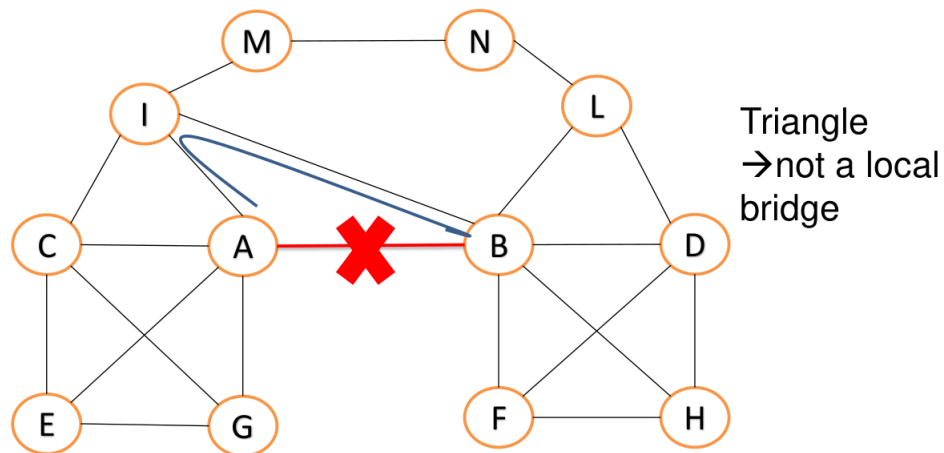
Per dare un teorema del perchè passo dal locale al globale introduciamo il concetto di strong triadic closure, ricordiamo che la triadic closure dice che se B e C hanno un amico in comune A allora è probabile che esista il legame tra B e C. La strong dice che se A e C e A e B hanno legami strong allora il legame B e C DEVE esistere. Chiaramente è un artificio matematico perchè nelle reti non vale sempre.

Oltre ad introdurre la strong triadic closure introduciamo il concetto di bridge, diciamo che un link tra due nodi A e B è un ponte se quando viene cancellato i nodi A e B appartengono a componenti differenti. Nelle reti sociali online sono iper connesse, quindi è molto raro che ci siano ponti, praticamente non succede mai.

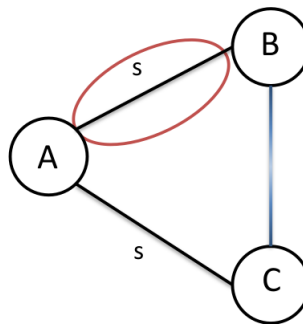


Dato che non troviamo veri ponti nelle reti sociali usiamo una definizione più blanda di ponte, diciamo che un link è un local bridge se due nodi endpoint non hanno amici in comune, in pratica se distruggo il link tra i due nodi il cammino minimo tra i due nodi diventa maggiore di 1.





Sotto la condizione che una parte della rete obbedisca alla strong triadic closure di dimostra che i ponti sono TUTTI link deboli. Questo è il punto in cui capiamo che la diade (proprietà locale) porta ad una triade (proprietà globale). Quindi sono i legami deboli che fanno da ponte tra le varie strutture forti della rete.



Dimostrare questa affermazione è facile, supponiamo per assurdo che dati due nodi A e B con un legame forte il legame sia un ponte, sicuramente il nodo A ha un legame forte verso il nodo C, ma siamo nell'ipotesi matematica della strong triadic closure quindi se ho due legami forti AC e AB allora sicuramente avrò il legame forte BC, abbiamo quindi dimostrato che AB non può essere un ponte (quindi legame debole).

### Almost Local Bridge

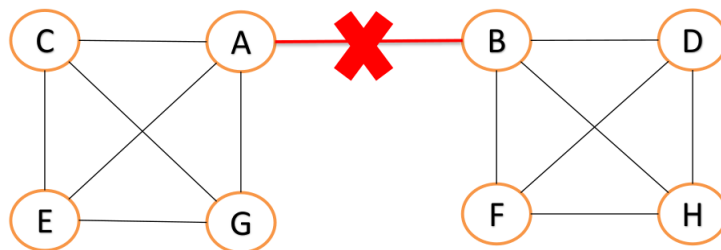
Andiamo ad approfondire meglio il concetto di bridge e il concetto di strutture forti. Abbiamo detto che i bridge forti nelle reti sociali non esistono, i local bridge sono difficili da trovare, si introduce quindi una nuova misura di quanto un link è un ponte. Si chiama almost local bridge, invece che usare la definizione

basata sui cammini si usa una definizione basata sul vicinato dei nodi. Si usa la neighborhood overlap, è il numero di nodi che sono vicini sia del nodo A e B diviso il numero di nodi che sono vicini o dell'uno o dell'altro:

$$\frac{\text{number of nodes who are neighbors of both A and B}}{\text{number of nodes who are neighbors of at least one of A or B}}$$

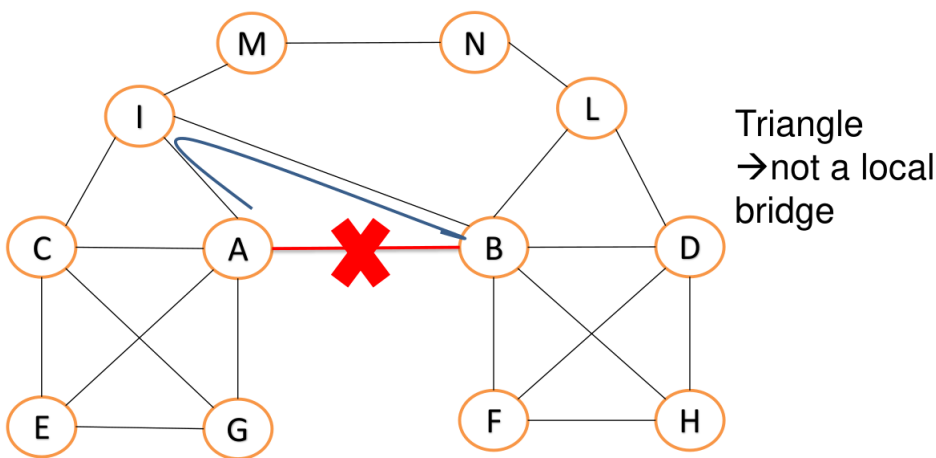
Prendiamo un ponte vero e proprio, A e B non hanno amici in comune, quindi l'overlap del loro vicinato è 0. Chiaramente più piccola è la sovrapposizione del vicinato dei due nodi, tanto più il link che connette i due nodi è un ponte, perchè connette nodi che non hanno vicini in comune.

$$O(A, B) = \frac{n(A) \cap n(B)}{n(A) \cup n(B)}$$



$$O(A, B) = 0$$

E' più significativa questa definizione rispetto alla local bridge, se ad esempio calcoliamo la sovrapposizione del vicinato di A e B troviamo che è  $\frac{1}{8}$  capiamo quindi che il legame non è forte.

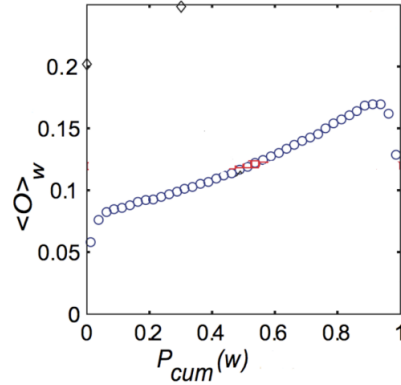


Quindi più è basso più il legame è debole, più è alto è forte. Il ponte si chiama almost local bridge perchè mette in collegamento due strutture di rete diverse.

Nelle reti sociali si usa la sovrapposizione del vicinato per il concetto di ponte.

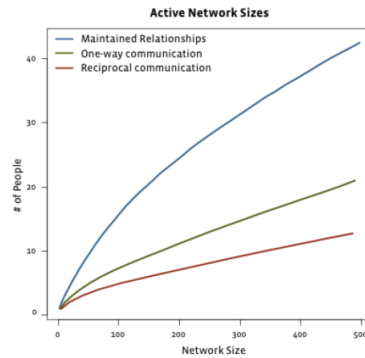
## Casi studio

### Mobile phone network



Hanno fatto uno studio della rete in cui i nodi sono dei numeri di una compagnia telefonica ed hanno un link se hanno almeno una chiamata tra di loro. Hanno deciso di misurare la forza del legame tranne la durata aggregata delle call. Potremmo usare come metrica anche la periodicità delle chiamate, possiamo usare quello che vogliamo ma si è dimostrato in alcuni studi che c'è una correlazione tra le metriche. Quindi come misurare la strength di un link dipende dai dati che abbiamo a disposizione. Detto questo in questo esperimento hanno deciso di usare la durata aggregata della call, plottano la neighborhood overlap dei link in funzione dei percentili della forza dei legami ordinata in senso crescente, vuol dire che considerano per ogni link la strength del link (la durata aggregata della call), poi prendo tutti i link che hanno la stessa durata aggregata e misuro la neighborhood overlap dei due nodi del link. Quello che si vede che al crescere della forza del legame cresce l'overlap della neighborhood. Quindi è vero che tanti più vicini abbiamo in comune tanto più è forte il legame che abbiamo. Letta in senso inverso tanto è più forte il legame che abbiamo tanti più amici abbiamo in comune.

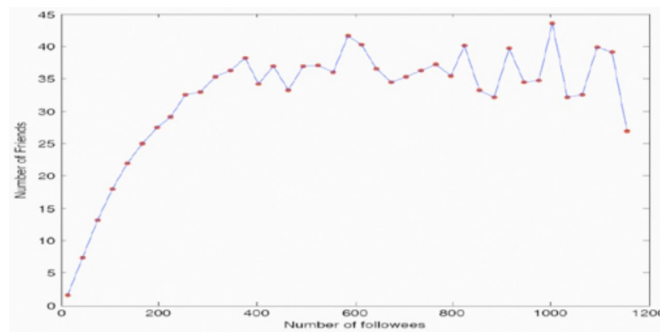
## Facebook



Sono partiti da questa domanda: le social network online favoriranno le relazioni? il fatto che sia un servizio diverso favorirà l'avere più relazioni? e che tipo di relazioni, forti o deboli? Lo studio è fatto su un mese ed i link vengono divisi in 3 categorie che corrispondono alla forza del legame tra due nodi, si dice che un link è una relazione mutual se entrambi gli utenti si scambiano a vicenda un messaggio, un link è detto one way se un utente manda un messaggio al secondo ed il secondo NON manda un messaggio al primo, poi ci sono le maintained relationship quando un utente segue un altro utente senza comunicazione diretta. Quindi hanno plottato un grafico che sull'asse delle x ha la network size di un utente (quante connessioni ha un utente), in funzione di questa che è la personal network di una persona, il numero di utenti che sono coinvolti in una relazione reciproca, one way o maintained. Lo studio vedeva dei pattern interessanti, le comunicazioni reciproche delle persone che hanno una personal network molto grande rimangono in numero limitato, le one way un po' di più e molto di più le maintained relationship.

Si scopre quindi che in realtà il numero di connessioni forti rimane limitato, aumenta il numero di connessioni deboli. Questo si lega ad un altro studio in cui si è dimostrato che in qualsiasi servizio ci viene fornito non abbiamo mai più di 50 relazioni strette.

## Twitter



E' lo stesso studio di Facebook, si è visto che nonostante il numero di follower cresce, il numero di amici satura.

## Sei gradi di separazione

Si chiama fenomeno dei sei gradi di separazione (small world) che per primo è stato studiato da un sociologo che voleva capire quanto le persone fossero vicine nella società umana, quanto due persone che non si conoscono, passando da amico ad amico in quanti passi possono raggiungersi. Prese delle persone a caso nel Nebraska che deve arrivare ad una persona a Boston, chiese a queste persone di passare una lettera (scrivendo il nome) per arrivare alla persona a Boston. Solo 64 di queste catene su 256 raggiunse il destinatario, è interessante notare che di queste catene chiuse il numero di hop necessari era stato 6. Quindi diciamo che con in media 6 passaggi, qualunque coppia di persone nella società umana riesce a comunicare.

Si è cercata di replicare questa cosa sui social, in particolare su Facebook. Il gruppo di ricerca ha sviluppato un algoritmo per calcolare i cammini minimi su reti grandi. Con questo algoritmo hanno scoperto che la distanza media tra coppie di nodi è 4. Nei vari studi si è visto che il fenomeno dei sei gradi di separazione avviene in ogni rete sociale. La lunghezza media dei cammini tra i nodi è sempre molto piccola. Questo da l'idea di come le reti sono interconnesse.

## Modello di small-world

Se pensiamo ad un nodo supponiamo che in media una persona conosca 100 altre persone (100 connessioni), se facciamo 1 hop diciamo che ogni persona conosce altre 100 persone, a due hop siamo già a  $100 \cdot 100 = 10000$ .

Stiamo dicendo che in una rete random, in cui abbiamo che ogni nodo ha numero di connessioni che chiamiamo valor medio del grado, sappiamo che un nodo a distanza 1 con 1hop raggiunge valor medio, a distanza due valor medio del grado al quadrato e così via. In generale in una rete random ogni nodo ha

valor medio del grado alla  $d$ . Il numero atteso di nodi raggiungibile dal nodo sorgente con un numero di hop minore di  $d$  è  $1 + \text{valor medio del grado} + \dots + \text{valor medio del grado alla distanza massima}$ . La massima distanza tra due nodi di una rete si chiama diametro della rete. Questo è quello che chiamiamo veramente small world.

Dato che nella frazione c'è il logaritmo sappiamo che scendiamo di molti ordini di grandezza.

Il diametro ha il problema degli outlier, quindi invece di considerare il diametro consideriamo la lunghezza media dei cammini e diciamo che la rete è small world se la lunghezza media dei cammini è il logaritmo del numero di nodi fratto il logaritmo del valor medio del grado:

$$\langle d \rangle = \frac{\log N}{\log \langle k \rangle}$$

Quello che si è visto è che calcolando questa quantità c'è un ottimo accordo tra quello che misuro sulla rete e quello che prevede la formula. Quindi la maggior parte delle reti è small world. Stiamo dicendo che qualunque tipo di rete consideriamo di sono delle proprietà generali.

## Reti sociali

Possiamo dire che le reti sociali sono small world per lo stesso motivo con cui l'abbiamo fatto nelle reti random, questo perchè nelle reti sociali abbiamo la triadic closure, non è vero che ogni nodo ha 100 amici diversi, la maggior parte saranno uguali. Ma allora perchè siamo small world? abbiamo un paradosso, abbiamo tanti triangoli ma cammini molto brevi, queste due cose a prima vista non possono convivere. La soluzione al paradosso sono i weak ties, i legami deboli uniscono circoli di nodi fortemente legati, quindi basta 1 weak tie per rendere ad 1 hop una parte di rete che altrimenti sarebbe molto lontana.

Questo modello di rete sociale è molto teorico, si è visto nelle reti sociali che tutto è diventato iper connesso.

## Watss - Storgatz model

vogliamo un modello generativo di rete che superi il paradosso di high clustering e di basso diametro.

Hanno inizialmente pensato di costruire una rete con un alto livello di clustering, mettono tutti i nodi su una circonferenza, ogni nodo è connesso ai  $c / 2$  nodi dall'altra parte della circonferenza. Poi cancello un legame forte a caso ed estraggo a caso una coppia di nodi nella rete e creo un legame tra quei due nodi, con questa operazione abbasso i cammini minimi fino ad arrivare al diametro che voglio.

Questo modello è chiamato, modello small world. Si è visto che il modello ben riproduce il coefficiente di clustering sia il diametro.



# Reciprocità

Misurare la strenght di un link non è facile, ne abbiamo l'intuizione ma poi cosa possiamo misurare? numero di interazione, la durata delle interazioni. C'è una misura significativa che è la reciprocità della relazione, una relazione reciproca è più forte. Calcolare la reciprocità permette in un grafo orientato quali link possiamo non considerare, perchè se abbiamo un link con una sole direzione probabilmente appartiene a due nodi che non hanno una relazione.

Se abbiamo una rete e vogliamo capire se è una rete sociale o una rete di informazione la reciprocità è una prima misura che ne rende conto, perchè tipicamente è presente nelle reti sociali. Quando abbiamo un grafo non orientato la reciprocità si concentra sulle diadi e sui loro link. Nei grafi orientati gli stati sono molteplici:

- Diade nulla: non ho link tra i nodi
- Diade simmetrica: ho un link tra entrambi i nodi
- Diade asimmetrica: ho un link solo da una parte

La reciprocità possiamo definirla come la percentuale delle diadi simmetriche tra le diadi non nulle. La percentuale delle coppie di nodi che hanno una relazione reciproca tra le coppie di nodi che hanno almeno un link tra di loro.

Esiste una seconda definizione, che si focalizza sui link, definisce reciprocità il numero di link che hanno un reciproco fratto il numero di link.

Consideriamo i link in una matrice di adiacenza dove se un link è non reciproco abbiamo degli zero, facciamo la sommatoria dei link, se uno è non reciproco significa che il numeratore va a 0, quindi è comoda la seconda definizione perchè in questo modo posso fare il quadrato della matrice di adiacenza e della matrice risultante la diagonale (traccia) sono i link reciproci.

## Teoria Balance and Status

L'idea è di capire le strutture mesoscopiche di tre link se sono stabili rispetto agli attributi che hanno sui nodi, per ora abbiamo visto strutture di rete in cui abbiamo nodi e link, ma chiaramente sui nodi e sui link possiamo avere informazioni aggiuntive, ad esempio sui nodi abbiamo gli attributi. Quando si hanno informazioni ulteriori rispetto alla pura esistenza dei nodi si parla di attributi.

La teoria del social balance cerca di fare un passo in avanti rispetto alla triadic closure (se tra due nodi ci sono due link è probabile che ci sia anche il terzo), qua da una valutazione del tipo di link che si ha in una relazione, perchè quando diciamo che in una rete c'è un link per ora diciamo solo che c'è una relazione tra i nodi, ma potremmo avere anche informazioni su questa relazione, che potrebbe essere la strenght o un attributo binario, una caratteristica positiva o negativa. Se su ogni link aggiungiamo anche che sia una relazione positiva o negativa (usando un attributo del link) possiamo arrivare alla teoria del social balance, che detta in modo informale sostiene che:

- L'amico del mio amico è mio amico (triadic closure). Abbiamo dato una connotazione ai link, i tre link sono positivi
- L'amico del mio nemico è mio nemico
- Il nemico del mio nemico è mio amico
- Il nemico del mio amico è mio nemico

Stiamo ancora parlando di transitività della rete (triangoli) ma nel momento in cui ho un informazione in più, l'attributo binario sulla relazione, la triadic closure diventa molto più complessa. Per modellare questo concetto su un grafo usiamo dei pesi sui link che possono avere solo due valori (-1 e +1) dove +1 rappresenta un link positivo e -1 se la relazione è negativa, queste reti si chiamano reti con segno. Diciamo che una struttura a triangolo in una rete è bilanciata se obbedisce ad una di queste quattro informazioni. Dal punto di vista analitico se il prodotto dei pesi dei tre link è maggiore uguale a 0 allora il triangolo obbedisce ad una di queste quattro regole.

La teoria dello stato sociale è l'idea che se un nodo A è più prestigioso di B e lo è più di C allora A lo è più di C. Possiamo sempre modellare la rete creando una rete orientata, dove la direzione del link indica che il nodo di partenza ha uno stato più basso di quello di arrivo, che equivale ad un link con segno +.



# Similarità

La similarità dei nodi una rete ha 3 possibili declinazioni:

- Due nodi sono simili se condividono la stessa struttura di rete, condividono gli stessi amici. Non basata su attributi.
- similarità per attributi, non deriva dal fatto di avere gli stessi amici, ma dall'avere simili attributi (scrivono nella stessa lingua, seguono gli stessi canali).
- Basata sui contenuti che pubblicano i nodi, usano gli stessi hastag

Quindi quando vogliamo misurare la similarità possiamo farlo usando tre informazioni diverse:

- la struttura di rete
- gli attributi dei nodi: assortatività
- i contenuti dei nodi: non la trattiamo

## Struttura di rete

Si chiama equivalenza strutturale il vicinato condiviso tra i due nodi, il numero di nodi dell'intersezione dei vicini definisce quanto sono simili i due nodi.

Si chiama vertex similarity l'intersezione del vicinato dei nodi  $v_i$  e  $v_j$ , non ci piace, perchè manca la normalizzazione.

La normalizzazione viene fatta in due modi:

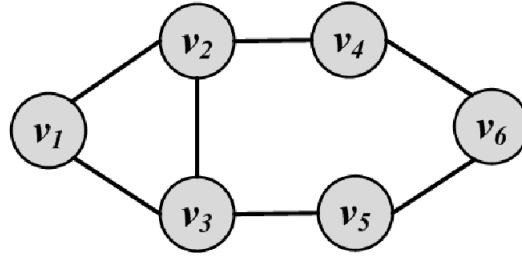
- Dividendo per la dimensione dell'unione del vicinato, questo si chiama coefficiente di Jaccard, che su qualsiasi cosa lavoriamo ci dice sempre un coefficiente di similarità.
- Cosine Similarity: dividiamo per la radice del numero di vicini di  $v_i$  per il numero di vicini di  $v_j$

$$\sigma_{\text{Jaccard}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|}$$

$$\sigma_{\text{Cosine}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{\sqrt{|N(v_i)| \cup |N(v_j)|}}$$

Dato che sono normalizzate il range è sempre 0,1.

Notiamo che è la stessa quantità del neighborhood overlap.



$$\sigma_{Jaccard}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{|\{v_1, v_3, v_4, v_6\}|} = 0.25$$

$$\sigma_{Cosine}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{\sqrt{|\{v_1, v_3, v_4\}| |\{v_3, v_6\}|}} = 0.40$$

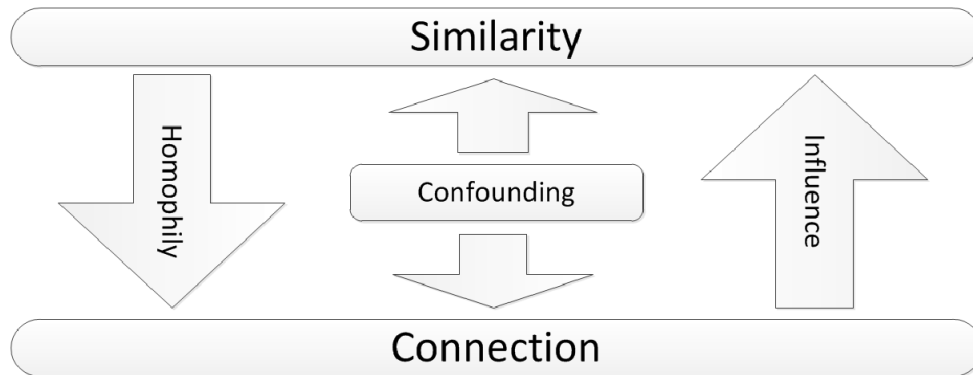
## Assortatività

E' la similarità sugli attributi dei nodi, mette insieme anche se tra i nodi c'è un link.

Si parte da una teoria sociologica che ha osservato come spesso nodi amici (tra cui c'è un link) hanno attributi simili. Notiamo nelle reti sociali che due nodi che hanno un link hanno una certa probabilità di avere attributi simili, dove per attributi intendiamo che possono avere interessi simili, attività simili... Si chiamano reti assortative reti in cui i nodi simili sono connessi tra di loro più che nodi dissimili. Parliamo di assortatività di una rete quando i nodi simili tendono ad essere connessi, più dei nodi dissimili o random. Visto dal punto di vista social osserviamo una grande similarità tra amici.

Ma perchè le persone che si conoscono sono simili? si sono visti tre possibili meccanismi:

- Influence: il processo per cui un individuo modifica un altro individuo in modo tale che il secondo individuo diventi più simile al primo.
- Homophily: il processo con cui due attributi che hanno gli stessi attributi creano un link
- Confounding: è l'impatto dell'ambiente, due nodi che vivono nella stessa città hanno più probabilità di formare un link rispetto a due nodi presi a caso nel mondo



In uno schema, se i due nodi sono simili per un processo di Homophily diventano connessi, il processo di Influence parte dalla connessione di due nodi e li fa diventare simili.

Ma alla fine anche se non sappiamo il processo che ha portato due nodi connessi da un link sono simili abbiamo una rete assortativa.

### Come misurare l'assortatività

La quantità che usiamo dipende se abbiamo attributi nominali o ordinale, è ordinale quando c'è una scala (un ranking, ad esempio l'età) mentre è nominale quando non posso dare un ordinamento tra gli attributi.

#### Attributi ordinali

Se abbiamo degli attributi ordinali voglio vedere la relazione degli attributi ordinali dei nodi con un link sto calcolando se i nodi connessi tra un link hanno una correlazione tra valori ordinali.

Costruisco due variabili  $x_{left}$  e  $x_{right}$  per ogni  $(v_i, v_j)$  edge e assumiamo che  $v_i$  è osservato dalla variabile  $X_L$  e  $v_j$  è osservato dalla variabile  $X_R$ . In altre parole le variabili  $X$  rappresentano i valori ordinali associati al nodo destro o sinistro.

Esempio: facciamo la lista dei link e per ogni link consideriamo anche l'orientamento (uno da una parte ed uno dall'altra), poi per ogni link vediamo quale nodo è a sinistra o destra ed associo l'attributo del nodo alla variabile  $x$  corrispondente alla direzione.

Saltando le formule scopriamo che la similarità si calcola con il coefficiente di correlazione, in particolare il coefficiente di correlazione di Pearson, che abbiamo già visto quando volevamo capire se due centralità sono correlate.

Sappiamo che il coefficiente di Pearson sta tra -1 e 1, in realtà quasi tutti i coefficienti di correlazione stanno tra -1 e 1 perché se una rete è assortativa ha un coefficiente positivo mentre abbiamo una rete disassortativa se negativo.

**List of edges:**

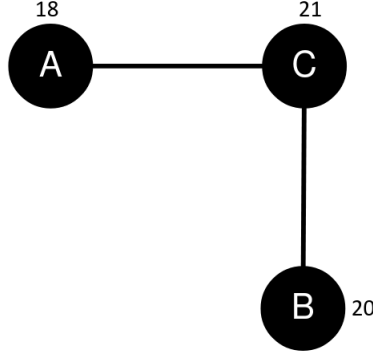
((A, C),  
(C, A),  
(C, B),  
(B, C))

- $X_L : (18, 21, 21, 20)$
- $X_R : (21, 18, 20, 21)$



$$E(X_L) = E(X_R),$$

$$\sigma(X_L) = \sigma(X_R).$$

**Attributi nominali**

Consideriamo reti non orientate  $G(V, E)$  dove  $n$  è il numero di nodi ed  $m$  è il numero di link, ed ogni nodo ha un tipo per l'attributo. Ad esempio se l'attributo che consideriamo è la lingua il tipo è la lingua stessa.

Calcoliamo la frazione di link tra nodi con lo stesso tipo, questo già di per sé mi dà un'idea di assortatività, mi dice se sono tanti o pochi. Il numeratore sono i nodi con lo stesso tipo

$$\frac{\sum_{(v_i, v_j)} \delta(t(v_i), t(v_j))}{m}$$

La funzione delta prende in input due numeri e ritorna 1 se sono uguali o 0 se sono diversi. La funzione delta ci filtra tutti i link e tiene quelli che hanno tipo uguale.

Cerchiamo di tradurre questa sommatoria scritta bene in senso matematico, scorriamo su indici e non su una lista di oggetti:

$$\begin{aligned} \frac{\sum_{(v_i, v_j)} \delta(t(v_i), t(v_j))}{m} &= \\ &= \frac{\sum_{(i, j=1)}^m A_{i,j} \delta(t(v_i), t(v_j))}{2m} \end{aligned}$$

In pratica scorro sugli indici, ma per vedere che esista il link tra  $i$  e  $j$  uso la matrice di adiacenza  $(A_{i,j})$ . Prendo solo le coppie di nodi che hanno un link tra di loro ed hanno lo stesso tipo danno un contributo positivo alla sommatoria. Divido per 2 per non contare ogni link due volte, dato che siamo in una rete non orientata.

Questa potrebbe già essere una misura di assortatività, perchè abbiamo già calcolato una misura che conta i nodi dello stesso tipo. Ma come misuriamo quanto è assortativa e quanto non lo è? manca il confronto con una rete random, per dire che una rete è assortativa devo dimostrare che vedo più link dello stesso tipo di quanti ne vederi in una rete costruita random.

Quindi calcoliamo la stessa quantità in una rete random, ma più sofisticata di quella di Reni. Perchè quella di Reni è veramente completamente random, però abbiamo imparato che le reti sociali sono scale free, con una distribuzione del grado molto lontana da quella random, quindi se vogliamo una rete random più simile rispetto a quella che stiamo studiando quella di reni non va bene. Usiamo un modello che ha lo stesso numero di nodi  $n$ , lo stesso numero di link  $m$  e la stessa distribuzione del grado. In questo modo la struttura della rete rimane molto simile rispetto a quella che stiamo studiando, questo per chiederci solo se l'attributo sul nodo guida dove sono posizionati i link. Dobbiamo calcolare il numero di link tra nodi dello stesso tipo che avrei in questa rete random.

Cominciamo a calcolare la probabilità di esistenza di un link tra due nodi  $v_i$  e  $v_j$  in questa rete. Questi nodi hanno un grado fissato, diciamo che il nodo  $v_i$  ha grado  $d_i$  e  $v_j$  ha grado  $d_j$ , quel'è la probabilità che esiste un link tra  $v_i$  e  $v_j$ ? Il nodo  $v_i$  ha di probabilità di formare un link, qual'è la probabilità che uno di questi abbia come endpoint finale  $v_j$ ? è il grado di  $v_j$ , quindi  $d_j$  diviso il numero di link (numero di nodi contati due volte):

$$p((v_i, v_j)) = d_i \frac{d_j}{2m}$$

Dobbiamo calcolare su questa rete random la frazione di link tra nodi dello stesso tipo:

$$\frac{\sum_{(v_i, v_j)} \frac{d_i d_j}{2m} \delta(t(v_i), t(v_j))}{m}$$

Quindi tra una rete random una non random cambia solo il fatto che nella rete non random abbiamo la matrice di adiacenza e nella rete random abbiamo la probabilità che esista il link tra  $v_i$  e  $v_j$ .

Poi di nuovo la formuliamo con gli indici:

$$\frac{\sum_{(i,j=1)}^n \frac{d_i d_j}{2m} \delta(t(v_i), t(v_j))}{2m}$$

L'assortatività è la differenza tra le due, quindi chiamiamo ASSORTATIVITA':

$$Q = \frac{\sum_{(i,j=1)}^m A_{i,j} \delta(t(v_i), t(v_j))}{2m} - \frac{\sum_{(i,j=1)}^n \frac{d_i d_j}{2m} \delta(t(v_i), t(v_j))}{2m}$$

l'assortatività è la differenza tra la frazione di link tra nodi dello stesso tipo e la frazione di link di nodi dello stesso tipo in una rete random in cui tengo lo stesso numero di nodi. Se questa differenza è positiva significa che la rete sociale è assortativa, quindi nodi dello stesso tipo tendono a formare link tra di

loro. Se invece la differenza è negativa significa che la rete è disassortativa. Se è intorno allo zero significa che non c'è una tendenza specifica.

La riscriviamo meglio:

$$Q = \frac{1}{2m} \sum_{i,j=1}^n (A_{i,j} - \frac{d_i d_j}{2m}) \delta(t(v_i), t(v_j))$$

- $Q > 0$  assortativa
- $Q = 0$  ne assortativa ne disassortativa
- $Q < 0$  disassortativa

Ma normalizziamo questa cosa, in modo da aspettarci sempre lo stesso range di risultati.

Normalizziamo rispetto al massimo:

$$Q_{normalizzata} = \frac{Q}{Q_{MAX}}$$

L'assortatività massima la raggiungiamo quando tutti nodi dello stesso tipo devono avere un link e non avelo con i nodi di tipo diverso. Quindi calcoliamo quanto vale questo massimo:

$$Q_{normalizzata} = \frac{Q}{\frac{1}{2m} \cdot 2m - \frac{1}{2m} \cdot \sum_{(i,j)=1}^n \frac{d_i d_j}{2m} \delta(t(v_i), t(v_j))}$$

$$Q_{normalizzata} = \frac{Q}{1 - \frac{1}{2m} \sum_{i,j=1}^n \frac{d_i d_j}{2m} \delta(t(v_i), t(v_j))}$$

Sappiamo che la assortatività normalizzata è sempre tra -1 e 1.

Rappresentazione matriciale dell'assortatività:

$$Q = \frac{1}{2m} \sum_{i,j=1}^n (A_{i,j} - \frac{d_i d_j}{2m}) \delta(t(v_i), t(v_j))$$

Introduciamo la matrice  $\Delta \in \mathbb{R}$  significa che ha un numero di righe pari al numero di nodi ed un numerodi colonne pari al numero di colonne, ed è definita in questo modo:

$$\Delta(x, h) = \begin{cases} 1 & t(x) = h \\ 0 & \text{altrimenti} \end{cases}$$

Così riusciamo a riscrivere la funzione delta in un modo matriciale:

$$\delta(t(v_i), t(v_j)) = (\Delta \Delta^T)_{i,j}$$

Prodotto della matrice stessa per la sua trasposta, solo quando abbiamo un elemento 1 otteniamo un 1 nella matrice moltiplicazione.

Possiamo riscrivere l'assortatività come:

$$Q = \frac{1}{2m} \sum_{i,j=1}^n (A_{i,j} - \frac{d_i d_j}{2m} \Delta \Delta^T)$$

Introduciamo il vettore del grado dei nodi, che è un vettore con n righe, ed una colonna, ogni riga è il grado del nodo.

Quindi definiamo la matrice B:

$$B = A - \frac{d \cdot d^T}{2m}$$

Quindi alla fine la formulazione matriciale dell'assortatività è:

$$Q = \frac{1}{2m} \text{Traccia}(B \cdot \Delta \cdot \Delta^T)$$

La traccia è la somma degli elementi della diagonale principale.

Quindi la rappresentazione matriciale normalizzata è:

$$Q_{\text{normalizzata}} = \frac{Q}{1 - \frac{1}{2m} (B^i \Delta \Delta^T)}$$

## Homophily

L'Homophily è la tendenza degli individui di associarsi con utenti simili. Se abbiamo dei dati temporali possiamo vedere come cambia l'assortatività, così facendo troviamo l'homophily.

## Influence

Per Influence in sociologica è l'atto di produrre un effetto senza un apparente forza. Come facciamo a misurarla, significa misurare quant'è l'influenza che un nodo esercita sulla rete, innanzitutto sui vicini e poi a cascata. Per misurarla possiamo usare due approcci.

Quello predittivo assume che il nodo ha delle caratteristiche che permettono di predire quanto sarà influente nella rete, tipicamente si parla di centralità. Tipicamente si usa l'in-degree, se un nodo ha tanti followe prediciamo che influenza facilmente la rete.

Oppure ho un approccio basato sull'osservazione, cerchiamo di vedere in un processo di influenza quanto un nodo ha portato i vicini ad avere lo stesso tipo. Abbiamo la size dell'audience e la size della cascata

## Casi studio

Non basta l'indegree, in questo lavoro sono andati più in profondità come concetto di influence, dobbiamo avere 4 caratteristiche per definire che un nodo è un influencer:

- Deve avere un riconoscimento dagli altri nodi, va bene l'in-degree
- Deve essere un nodo le cui attività portano ad avere attività degli altri nodi come follow up
- Deve avere un grado di novità, deve proporre qualcosa di nuovo
- Deve essere influente

Misuriamo la recognition come in degree, per riconoscere la caratteristica di influenza riguardo la generazione di attività di chi ti segue la misuriamo come numero di commenti. La novità la misuriamo tramite gli out link, con l'idea che se stiamo scrivendo qualcosa di nuovo non abbiamo out link verso nient'altro, tanto più ho out link tanto meno è la novità che propongo. L'eloquenza è la lunghezza del post, tanto più lungo è il post tanto più dico che l'influencer pubblica in modo eloquente. E' solo una proposta, i modi per misurarli possono essere diversi.

### Twitter

L'in-degree ci serve per vedere il riconoscimento verso quella persona, possiamo usare il numero di mentions per capire se un utente ha una buona engagement, mentre il numero di retweet da un'idea di quanto un nodo è influente perché riesce a generare un contributo che gli altri nodi pensano valga la pena trasmettere agli altri nodi della rete.

Tutte queste cose possiamo usarle insieme, usare dei pesi o usarle singolarmente, dipende da cosa vogliamo studiare.

Tipicamente quello che si nota è che l'in-degree non ha una correlazione tanto alta con il numero di mention e di retweet, avere tanti follower non vuol dire essere veramente un influencer, vale di più avere alto il numero di retweet e di mentions.



# Community Detection

Ci manca una misura che rende conto com'è fatta la rete a grandi pezzi, ma non grandi come le componenti connesse. Significa identificare gruppi di nodi (link tra di loro), che sono a metà tra i coefficienti di clustering e le componenti connesse.

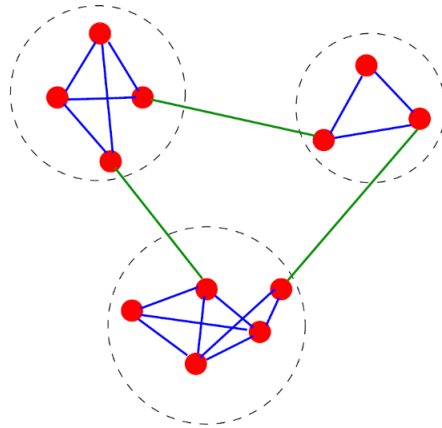
Perché studiare le comunità? le comunità le pensiamo come grandi gruppi di persone che condividono un interesse, che sono molto legate tra di loro. Ovviamente identificare le comunità in una rete è importante perché significa trovare quelle strutture di rete in cui gli individui hanno un forte legame tra di loro. Quando studiamo i link tra poche persone portiamo dentro un sacco di rumore, invece se riusciamo a dare una struttura della rete è meglio. Inoltre tendenzialmente i gruppi sono più stabili, cambiano più lentamente rispetto alle singole strutture. Nelle social network si hanno due tipi di comunità:

- Gruppi espliciti: i membri del gruppo esplicitano la loro appartenenza al gruppo
- Gruppi impliciti: sono i gruppi i cui membri non hanno effettuato un iscrizione, non c'è nella rete il fatto che il gruppo si dichiara essere un gruppo. Insieme di nodi che senza fare un iscrizione (no esplicitato), sono un gruppo perché la struttura di rete che formano rende conto del fatto che sono molto legati tra di loro. Per esempio se prendiamo la rete sociale dell'università una classe è un gruppo implicito.

Questi gruppi si possono chiamare anche cluster, ma non facciamo così altrimenti facciamo casino con il cluster del machine learning. Google ebbe l'intuizione per primo che il social newtorking voleva dei gruppi, quindi creò Google+, ma fallì. Quindi l'idea di gruppi ha lanciato tantissime social network, mentre ha ucciso altre.

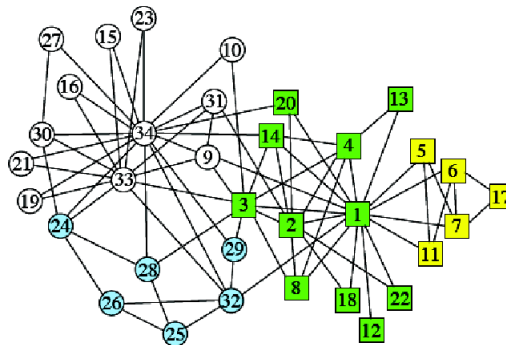
I gruppi espliciti ci interessano fino ad un certo punto, quando parliamo di communityDetection intendiamo trovare in una rete le comunità implicite. Quindi basandomi solo sul grafo formato dai nodi con le loro relazioni vogliamo trovare le comunità nascoste implicite nella rete. Sotto c'è che la struttura di comunità della rete è scritta nel grafo della rete, il fatto che ci siano gruppi di persone molto legate tra di loro si riflette sul grafo e noi possiamo identificarle.

Perché pensiamo che in una rete sociale ci sono le Community? per quello che abbiamo visto con i bridge, l'idea è che intorno ad ogni nodo c'è una ego network, ma poi allargandoci l'idea è che non solo ogni nodo ha un gruppo coeso di nodi vicino a se, ma fa parte di un gruppo di nodi che hanno caratteristiche comuni e questi gruppi grazie alle weak ties si collegano con tutta la rete.



Studiando la rete di collaborazione del Santafe Institute si è fatto un algoritmo di communityDetection ed il risultato è la rete in cui le Community sono identificate da colore dei nodi diverse, quello che si è visto è che si sono trovati gruppi di nodi che effettivamente sono Community perchè sono delle suddivisioni dell'istituto di ricerca. Quindi effettivamente in molte reti sociali c'è una struttura di Community che sta sotto la rete e può essere identificata tramite la community detection.

Vediamo l'esempio dello Zachary's Karate Club, l'allenatore ed il manager litigano ed il club si separa in due parti, per capire come si separano hanno chiesto agli individui gli altri membri con cui uscivano fuori dallo sport, si è visto che si sono create 2 Community ed effettivamente il club si è diviso secondo queste due community.



Una divisione netta si è vista nella politica, essendoci una grossa componente nella politica si trovano community ben definite e senza grosse sovrapposizioni. Quindi il fatto che si riesca a definire bene delle comunità dipende dal tipo di rete, se una rete è molto social è difficile trovare delle distinzioni nette.

Quindi si sono definite due tipi di Community:

- Separate: ogni nodo può appartenere ad una ed una sola community

- Sovrapposte: un nodo può appartenere a più community

Dobbiamo quantificare le cose, e per farlo dobbiamo fornire una definizione univoca di quello che stiamo cercando ed in questo momento cerchiamo una definizione di community, ma non esiste una definizione univoca di community, stiamo quindi parlando di un problema mal posto. Quindi possiamo dare tante definizioni diverse, per ogniuna di queste trovare delle metriche e degli algoritmi. Questo è il grande problema della community detection, quindi quando parlo di algoritmi di community detection parliamo di euristiche.

La prima intuizione è che ci devono essere più link all'interno della community rispetto ai link tra la community ed il resto della rete. La trasportiamo nel linguaggio delle reti dicendo che una community è un sottografo connesso localmente denso (la densità è maggiore dentro al gruppo rispetto a fuori) all'interno di una rete. Se vogliamo andare più in fondo possiamo dare almeno due ipotesi di cos'è una comunità:

- Ipotesi di connettività: la comunità è un sottografo connesso
- Ipotesi di densità: significa che i nodi all'interno di una community hanno probabilità più alta di fare link tra di loro rispetto ai nodi al di fuori

Questo è ancora molto vago però.

## Cliques Massimale

Una prima idea di definizione è che una community è una cliques massimale (un gruppo di nodi in cui c'è almeno un link tra tutti i nodi). Non è una buona definizione, innanzitutto nelle social network si trovano tantissimi triangoli, quindi clique massimali con 3 nodi, già trovare clique più grandi è difficile. Sostanzialmente è un concetto troppo restrittivo. Inoltre trovare le Cliques è un problema NP-completo, computazionalmente molto forte. Allora si è cercato di rilassare il concetto di clique, e qui sono nati tutti i concetti di clique rilassate, si è rilassata l'idea che tutti nodi abbiano un link verso tutti gli altri.

## Partizioni

Dobbiamo usare un altro approccio, usare degli algoritmi che trovano dei gruppi di nodi abbastanza grandi e coesi tra di loro.

La partizione di una rete è una divisione della rete in un insieme di gruppi tali per cui ogni nodo appartiene ad un solo gruppo.

- Community Detection: il numero di community e la grandezza delle stesse non è definita a priori. E' la scrittura della rete stessa che mi dice quali sono le community
- Partition Detection: il numero e la grandezza dei gruppi è fissata a priori. Questa si fa nei datacenter dei social network.

Devo trovare una definizione di community ed algoritmi che senza fissare e priori le dimensioni mi trova le community.

## Modularity

Ci sono vari approcci, dobbiamo trovare una definizione per trovare le community da una struttura di rete, abbiamo in mente che in una rete random non c'è nessuna comunità, mentre se ho una rete che ha delle community, quello che mi aspetto è di trovare regioni di una densità maggiore rispetto a quella che osserverei in una rete random. Quindi stiamo dicendo che una community possiamo vederla come un sottografo che ha una densità maggiore di quella che vedrei in una rete random. Chiamiamo quanto la densità di un sottografo cambia tra la nostra rete e la corrispondente rete random, modularità, è l'eccesso di densità in un sottografo rispetto alla rete random misura quanto è buona la struttura di community che ho nella mia rete. Se partizioni in community e misuro la modularità, tale modularità mi dice quanto la densità è maggiore rispetto a quella di una rete random, mi dice quindi quanto è buono il mio partizionamento.

Daltro lato stiamo dicendo che tanto maggiore è la modularità del mio partizionamento in community tanto meglio ho partizionato, quindi è una quantità che se massimizzata mi trova la partizione ottima. Questo è esattamente la stessa quantità che abbiamo visto nell'assortatività, solo che invece che usare l'attributo dei nodi usiamo che appartengono alla community. E' come se dessi ad ogni nodo un'etichetta, che è il nome della community a cui appartiene secondo l'algoritmo di community detection. La funzione delta invece che essere calcolata come 1 se i nodi hanno lo stesso attributo è 1 se appartengono alla stessa comunità. Tanto è vero che quella che abbiamo chiamato assortatività è anche chiamata modularità rispetto ad un attributo.

$$Q = \frac{1}{2m} \sum_{i,j=1}^n (A_{i,j} - p_{ij}) \delta(t(v_i), t(v_j))$$

Diciamo che se in una rete sociali calcolo la differenza tra il numero di link in una community ed il numero di nodi in un grafo random sugli stessi nodi e trovo un valore alto significa che avrò una modularità alta e quindi ho fatto una partizione buona della rete.

In altre parole definisco la modularità come un eccesso di link in un sottografo rispetto ai link che avrei in una rete random.

Il range è tra  $-\frac{1}{2}$  e 1. Se consideriamo tutto il grafo come community il valore della modularità è 0, se invece troviamo valori minori di zero significa che non siamo riusciti a partizionare correttamente la rete. Quindi in realtà gli unici valori validi sono quelli positivi.

## Community Detection Method

Ci viene in mente che un algoritmo possibile è trovare la partizione con massima modularità, cerchiamo quindi un algoritmo euristico che cerchi la partizione con massima modularità.

### Greedy Algorithm

L'algoritmo greedy è un algoritmo che si basa sull'idea: troviamo la partizione che dà la massima modularità tramite un algoritmo greedy.

- Start: ogni nodo è una community, la modularità è  $< 0$
- Per ogni coppia di community che hanno almeno un link tra di loro calcoliamo quanto guadagnerei in modularità se li unissi in un'unica community. Ripeto questi due passi.
- Arrivo fino a che tutta la rete è una community. Però ad ognuno degli step di unione registro quanto è la modularità ed alla fine dico che la partizione migliore è quella che ha la modularità massima.

Ad ogni passo cerca di unire due community, faccio il merge se ho il massimo guadagno di modularità. Ma attenzione che la modularità ha tanti massimi locali, quindi l'algoritmo greedy troverà una partizione subottima (trova la modularità massima locale), ma comunque buona.

## Community Detection On Networks

L'obiettivo di questi algoritmi è identificare gruppi di nodi identificati con la proprietà di omogeneità, ovvero simili tra di loro. Inizialmente non si usavano proprietà diverse dalla topologia della rete, in sostanza data una network senza nessun attributo sui nodi vogliamo identificare dei gruppi che sono simili tra di loro. Da un lato è un problema cardine all'interno dell'analisi delle reti, dall'altro è un problema mal posto, non esiste una definizione universale per community, di conseguenza abbiamo tantissimi metodi per l'estrazione di gruppi.

Abbiamo quindi tantissimi algoritmi, partiamo da quelli semplici ed intuitivi.

Identificare gruppi è un problema così vasto che è meglio sapere cosa possiamo utilizzare in base al contesto, alcuni algoritmi funzionano solo su grafi orientati, alcuni sfruttano il peso associato agli archi, alcuni non prendono in considerazione i pesi anche se ci sono, alcuni funzionano quando ho grafi con più livelli (più tipi di relazioni fra i nodi). Devo anche considerare l'approccio che utilizzano per identificare le community, alcuni utilizzano un approccio top-down, partono dalla rete generale e poi identificano i gruppi (parto dall'intera rete e fraziono sempre di più), l'approccio contrario è il bottom-up, considero ogni nodo come una singola community ed unisco gruppi più simili fra di loro. Possiamo anche definire vari tipi di community:

- Hard: un nodo appartiene ad un solo gruppo
- Overlapping: gruppi di nodi che si sovrappongono
- Fuzzy: danno le probabilità di appartenenza alla community
- Nested: i gruppi non sono omogenei, alcuni nodi hanno relazioni più strette, ci sono delle strutture a community innestate
- Hierarchical

Per ogni nodo possiamo associare più community, alcuni algoritmi permettono di identificare questi comportamenti.

Come dicevamo prima non esiste una definizione universale di community, ma diciamo che consideriamo una community come un insieme di nodi, quindi qualora utilizzassimo un approccio di tipo hard, in quel caso parleremmo di partizioni (prendiamo un insieme e lo partizioniamo in gruppi disgiunti). L'altro concetto è che ogni nodo è più vicino agli elementi che appartengono allo stesso insieme rispetto agli elementi di un altro insieme, questo va a catturare l'idea di densità e compattezza.

In community ristrette la visualizzazione ci permette di fare facilmente community detection, utilizzando la force directed layouts (se due nodi hanno un link si attraggono).

In reti grandi non è possibile farlo, dobbiamo quindi usare degli strumenti automatici per la visualizzazione delle community.

Proviamo a dare delle ipotesi:

- La struttura delle community dipende solo dalle relazioni espresse dalla network. Non uso gli attributi sui nodi, eventualmente li uso dopo
- Una community è un sottografo connesso: per concetto di densità tutti i nodi devono essere connessi
- Le community sono vicinate localmente densi in una rete: densità locale, se prendo il vicinato di una community quei nodi sono connessi tra di loro

## CD by Bridge Detection

Ci basiamo sul concetto di bridge, il bridge dovrebbe connettere delle community, perchè è quella zona in cui inizia a scemare il concetto di densità. L'idea è identificare i bridge, rimuoverli in modo da identificare le componenti (connesse) ed ogni componente corrisponderà ad una community. Posso agire con questo processo di taglio degli archi andando a tagliare tutti gli archi.

Il primo algoritmo che sfrutta questa idea è quello di Girvan Newman, l'idea è che la betweenness centrality ci permette di identificare un bridge partendo dai nodi. Prendono la network, calcolano la betweenness centrality degli archi e tolgono l'arco con betweenness centrality più alta e lo tolgono dalla network. Così via fino a che voglio, posso identificare il numero di community obiettivo oppure vado avanti a tagliare finchè non ci sono più archi.

Questo perchè posso rappresentare il processo di taglio, che Inizialmente è top down, possiamo generare il dendrogramma, è la visualizzazione del processo inverso che fa l'algoritmo, parte da tutti i nodi singoli e pensa ogni taglio come l'inserimento di un link, fino ad arrivare alla network. Partendo dal basso ogni community è un nodo, quando ho la connessione tra due nodi significa che si va a creare una community.

C'è un raffinamento, il metodo di Radicchi, che non usa le betweenness centrality, ma usa l'edge clustering coefficient di un arco:

$$C_{i,j}^3 = \frac{z_{i,j}^3 + 1}{\min(k_i - 1, k_j - 1)}$$

Prendo quello che ha l'edge clustering coefficient più basso, significa che intorno a lui non c'è niente, non ci sono triangoli, senza di lui i due sottografi si allontanano. Scorro il numero di archi e vedo in quanti il lato compare, diventa circa lineare la complessità.

## CD by Internal Density

L'approccio che vediamo adesso è quello di gruppo denso, vuol dire che il partizionamento che devo cercare deve essere denso tra gli elementi del partizionamento e meno denso se considero le connessioni tra gli elementi del partizionamento e quello che c'è fuori. Utilizziamo la modularità come funzione di qualità, dato un partizionamento ci dice quanto quel partizionamento si avvicina alla qualità desiderata, più si avvicina ad 1 più sto raggiungendo un partizionamento ottimo.

$$Q = \frac{1}{2m} \sum_{ij} [A_{ij} - \frac{k_i k_j}{2m}] \delta(c_i, c_j)$$

$$\sum_{ij} A_{ij} \delta(c_i, c_j) - \sum_{ij} \frac{k_i k_j}{2m} \delta(c_i, c_j)$$

La prima sommatoria rappresenta la densità della network, la seconda la densità di una rete random, se la differenza è positiva significa che c'è un senso nell'aggregazione in community, altrimenti è casuale.

Esiste un algoritmo facile, partizioniamo inizialmente in due community a caso, prendiamo un nodo e vediamo se è conveniente fare il cambio di gruppo, semplicemente calcolo la modularità cambiando gruppo, se cresce cambio gruppo e non lo tocco più, poi rifaccio il ragionamento per tutti gli altri nodi diversi da quelli che ho già valutato. Se non ci sono passaggi mi fermo.

## CD by Label Propagation

Se fossero gli altri nodi a dirmi in che community sono? l'idea viene esplicitata dall'algoritmo di label propagation, che è uno degli algoritmi più scalabili.

Innanzitutto partiamo da un grafo non orientato dove ogni nodo è una community (gi identifica la community data al nodo  $i$ ) ogni nodo ha come community il suo id. Per  $k$  volte, finchè le etichette non cambiano, ripetiamo: ogni nodo  $i$  adotta l'etichetta più frequente tra i suoi vicini.

Nella prima notazione selezioniamo tramite la matrice di adiacenza l'etichetta che massimizza il conteggio,  $g_j$  è l'etichetta del mio vicino,  $g$  è l'etichetta attuale, in pratica calcolo la frequenza di tutte le etichette possibili tra i miei vicini e svegliamo quella più frequente.

Nel caso di grafo pesato non orientato è un meccanismo di voto con il peso. Risoluzione di pareggi tra le frequenze:

- Scelta casuale tra i candidati
- Lable Inclusion strategy: aggiungo un self loop ad ogni nodo
- Lable retention strategy: se una delle più frequenti è quella attuale la seleziono, altrimenti scelgo in modo casuale

L'algoritmo che abbiamo descritto ora agisce in una maniera sincron, allo stesso istante tutti i nodi fanno l'aggiornamento, in sostanza tutti i nodi utilizzano le lable al passo precedente rispetto all'aggiornamento. Se uso una propagazione sincrónica diventa instabile, ad ogni iterazione cambia completamente il partizionamento, cambiano le etichette.

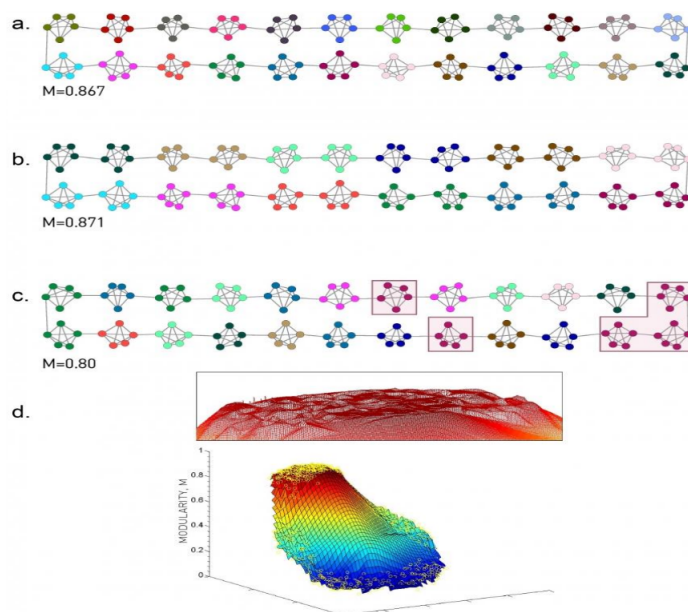
Per ovviare a questo problema si usa la propagazione asincrona, le etichette dei nodi sono aggiornate in modo sequenziale ma con ordianamento casuale, se sono prendo sempre l'aggiornamento più fresco, ognuno si attiva uno dopo l'altro, ovviamente l'ultimo nodo che si aggiorna è quello che vede la tutto più aggiornato.

Il vantaggio di questa implementazione è la complessità lineare.

## Limiti risoluzione massimizzazione modularità

I limiti di usare la modularità è che identificano community molto grandi. La superficie di modellazione con la modularità ha tantissimi massimi locali che corrispondono a delle grosse community. Un modo per ovviare a questa cosa è fare un primo run dell'algoritmo, trovare le community e rifare community detection in ogni community trovata per trovare community più vicine al concetto di comunità che vogliamo esprimere.

Inoltre, come dicevamo la modularità nel suo massimo ha una superficie con tanti massimi globali, quindi è difficile trovare il massimo assoluto.



L'algoritmo greedy non risolve questo problema con la modularità.

## Louvain algorithm

E' sempre un algoritmo di community detection non sovrapposte che punta a massimizzare la modularità mitigando il problema dei molti massimi e della scalabilità (è capace di affrontare reti con milioni di nodi).

Si applica anche alle reti con pesi, per ora vediamo solo sulla struttura di rete. Anche questo algoritmo è agglomerativo.

### Fase 1

- Come inizializzazione parte come l'algoritmo greedy, assegna ad ogni nodo una community, ho  $n$  nodi ed  $n$  community.

- Per ognuno dei nodi nella rete consideriamo i suoi vicini, valutiamo quanto guadagneremmo in modularità se rimuoviamo il nodo  $i$  nella sua community e lo mettiamo nella community più vicina. Mettiamo il nodo nella community vicina in cui avrei il massimo guadagno di modularità. Se non avessi nessun guadagno di modularità il nodo rimane nella sua community. Questo è un processo estremamente locale.
- Itero su tutti i nodi della rete finchè non ho più guadagno di modularità per tutti i nodi. Posso iterare sui nodi più di una volta.
- Con questa iterazione già raggiungiamo un massimo locale, perchè se ci fermiamo significa che non c'è modo di ottenere un'altro massimo.

Con questa iterazione abbiamo un massimo che dipende dall'ordine con cui prendo in considerazione i nodi. Quindi se prendo un ordine diverso potrei ottenere un valore di massimo locale diverso. In realtà su reti reali si è visto che l'ordinamento non ha un forte impatto sul risultato, perchè per ogni iterazione si trovano valori di massimo locale molto vicini.

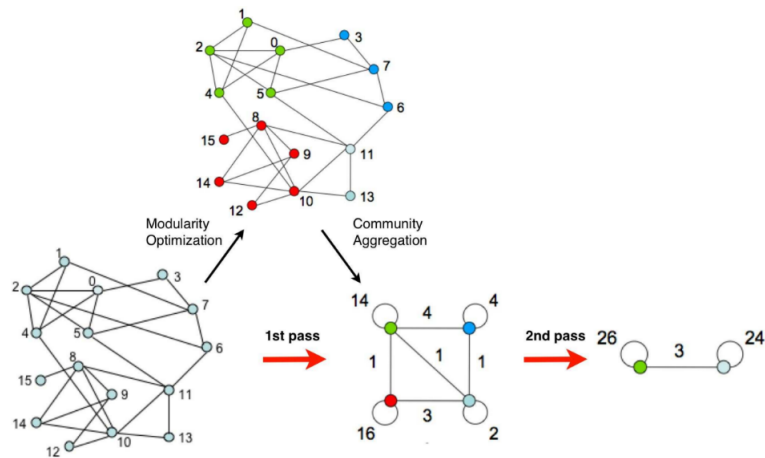
## Fase 2

Costruiamo una nuova rete, i cui nodi sono le community identificate nella fase 1. I link tra questi nuovi nodi sono la somma dei pesi dei link tra i nodi che avevo nella community. Ogni nodo che ho in questa nuova rete metto un self loop tra ogni nodo, il cui peso corrisponde alla somma dei pesi dei link tra la community (per non perdere l'informazione che ogni community aveva dei link tra i nodi che la componevano).

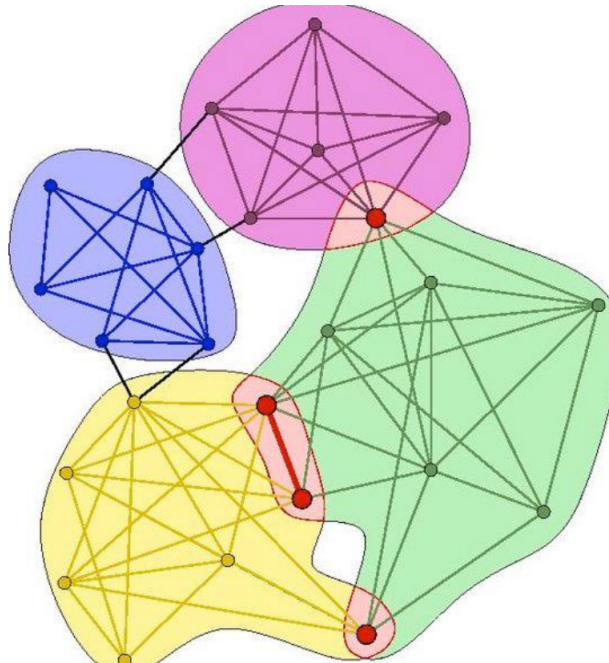
Una volta che abbiamo completato la fase 1 e la fase 2 le ripetiamo in sequenza. Ogni iterazione la chiamiamo pass. Ripeto le pass finchè non ho più nessun guadagno di modularità. Chiaramente ad ogni pass il numero di community decresce.

## Efficienza

effettivamente è un algoritmo efficiente, la modularità è una funzione che va calcolata sull'intera rete (la modularità è la somma per  $i$  e  $j$  della matrice di adiacenza meno il valore atteso dei link che avrei in una rete random per la funzione delta della coppia di due community diviso  $2m$ ) che è computazionalmente molto pesante da calcolare ad ogni passaggio. Si dimostra che con questo algoritmo dato che è locale, dovendo calcolare il guadagno in modularità può essere calcolato in modo molto semplice, perchè implica il calcolo solo sui nodi vicini.



## Overlapping Community



Come mai nelle reti sociali abbiamo community che si sovrappongono? perchè noi come umani abbiamo cerchie di amici che poi si sovrappongono.

Per fare Overlapping community detection dobbiamo partire da una struttura di rete rigida e fortemente connessa:

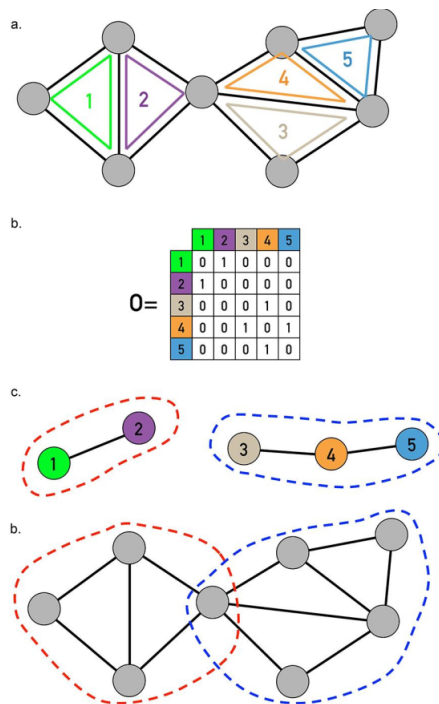
- Costruiamo le comunità sovrapposte partendo da gruppi fortemente coesi,

le clique. Due  $k$ -clique (sottografo completo di  $k$  nodi). Indichiamo due  $k$ -clique adiacenti se condividono almeno  $k-1$  nodi.

- Chiamiamo  $k$ -clique community il più grande sottografo connesso ottenuto dall'unione di tutte le  $k$ -cliq adiacenti.
- L'idea è di costruire le community aggregando le  $k$ -clique adiacenti.

L'algoritmo si chiama CFinder:

- primo step l'identificazione di tutti i triangoli (nelle reti sociali siamo sicuri che abbiamo tanti 3-clique, per il fenomeno della triadic closure). Le trovo e le enumero, faccio la lista di tutte le 3-clique che ho nel grafo.
- Poi costruisco una nuova rete i cui nodi sono le clique e dico che esiste un link tra i due nodi della rete delle clique se le due clique sono adiacenti. Questo vuol dire che posso costruire la matrice di adiacenze della rete delle clique, ho 1 se le due clique sono adiacenti.
- Su questa nuova rete calcolo le componenti connesse. Le community del grafo di partenza sono le componenti connesse del grafo delle clique.



L'esempio lo hanno fatto con le telefonate in Belgio, si è visto che la maggior parte delle telefonate di un gruppo rimane all'interno del gruppo (tipicamente diviso per lingua).

Si plotta sempre la size delle community, calcolo per ogni community il numero di nodi e poi ne faccio la distribuzione.

# Machine Learning

Vedremo degli algoritmi di machine learning per poter poter rispondere ad ulteriori domande di ricerca.

Partiamo da alcune definizioni generiche:

- Un agente software si dice apprendere dall'esperienza  $E$ , apprende rispetto ad un task  $T$ . Per capire se stiamo apprendendo dobbiamo avere una misura di performance che dovrebbe crescere man mano che l'esperienza aumenta.

Un approccio pre machine learning poteva essere creare un insieme di regole ed applicarle a dei dati e miglioravamo le performance delle regole migliorando le regole, quindi non c'è un apprendimento del software. Ma se le regole sono troppe ogni sforzo sarà vano. L'altro vantaggio è che se i dati che mi arrivano non sono stazionari (cambia la modalità con cui i dati sono generati).

3 criteri per categorizzare il problema da risolvere con il ML:

- Posso avere una supervisione umana? c'è qualcuno che mi dice se i risultati ottenuti sono giusti?
- Possono imparare on the fly?
- Che tipo di approccio uso per definire il modello? uso dei dati reali (instance base) o definisco un modello di realtà e cerco di identificare i parametri di quel modello che meglio descrivono la realtà?

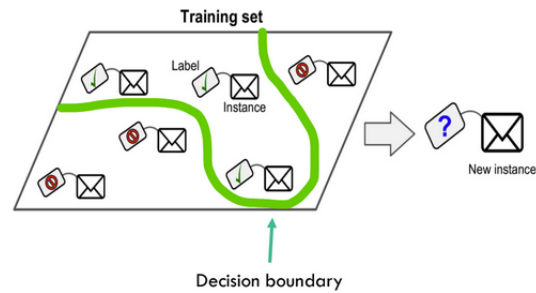
## Utilizzo supervisione umana?

Se utilizzo supervisione si utilizzano modelli di supervised learning, ho una supervisione del processo di apprendimento (c'è qualcosa che mi dice se stiamo sbagliando o no), quella cosa è l'etichetta. Ogni elemento dell'insieme di dati ha un etichetta.

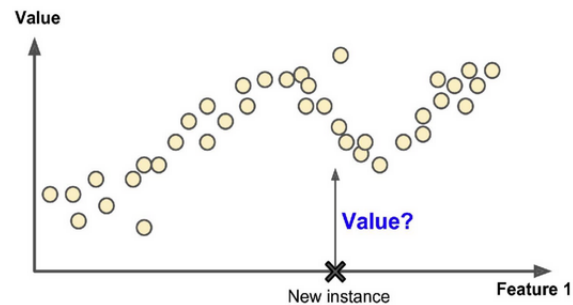
- KNN
- Linear and logistic regression
- Support vector machine
- Decision Trees and Random Forests
- Neural Networks

Se le etichette sono discrete e non ordinate si parla di classificazione. Se il numero di etichette sono 2 è classificazione binaria, se sono maggiori di 2 è multi class classification. Quello che imparo è la linea verde (decision boundary, iperpiano di separazione) che mi permette di separare dei punti (spam) da altri punti (non spam).

Se le etichette sono continue si parla di regressione, dato un insieme di punti assegnare ad ogni punto un valore appartenente alla retta reale. (stimare il

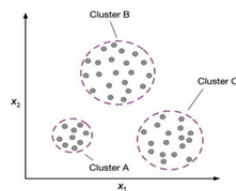


prezzo di una casa in base alle caratteristiche). Si parla di variabili indipendenti per identificare le proprietà di un oggetto e di variabile dipendente per identificare l'etichetta assegnata all'oggetto.

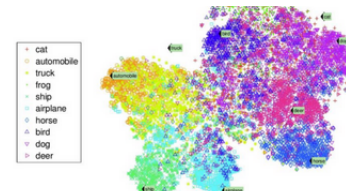


Se non abbiamo le etichette siamo in un contesto di unsupervised learning, per imparare uso le relazioni che esistono tra i dati che ho, cerco di definire un concetto di similarità fra i dati e sfruttare la similarità per trovare delle strutture significative. La community detection fa questa cosa, la similarità è il collegamento tra due persone, e da questo ne abbiamo estratto strutture significative.

- Clustering: organizziamo i dati in gruppi significativi. Per fare questa cosa devo definire un concetto di similarità.
- Riduzione della dimensionalità: solitamente gli elementi che abbiamo a disposizione sono descritti da un numero di proprietà variabile, gli oggetti possono essere descritti da molte proprietà, quindi molte dimensioni, l'obiettivo è ridurle.
- anomaly detection: verificare se c'è un comportamento anormale nei dati
- Association rules: identificare le regole più presenti



Clustering

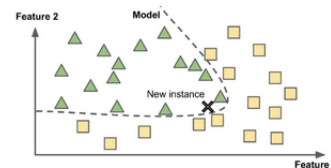


Dimensionality reduction

## Che tipo di modello ho?

Questo ci porta alla domanda cardine del machine learning, ho un modello che è stato addestrato dall'esperienza dai dati visti, se fornisco un nuovo dato al modello, questo è in grado di dirmi correttamente l'etichetta? Dobbiamo permeromare bene su dati non ancora osservati. Si parla di overfitting, ha imparato a memoria il sistema di dati di addestramento ma non è capace di generalizzare.

- Instance base: tengo in memoria tutti i punti che ho già visto ed ogni punto nuovo lo associo a dei punti che avevo già in memoria.
- Model based: devo tirare una linea per dividere i punti. Quando arriva un nuovo punto chiedo al modello da che parte della riga sta il punto

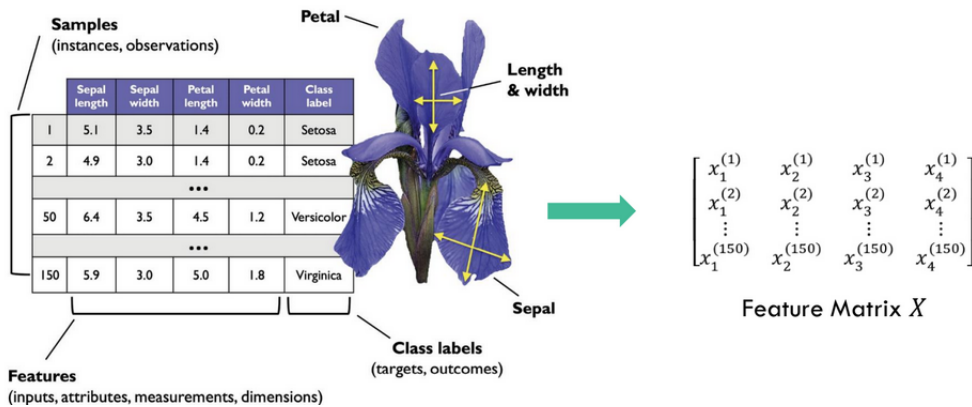


Cosa può andare storto? ho pochi dati, i dati non sono rappresentativi (ho un insieme ridotto di dati di apprendimento), sampling bias (come vado a collezionare i dati), feature selection, data gatering. Underfitting: il modello fa schifo, anche sui dati di addestramento il modello non apprende. Overfitting: il modello va bene sul training set, ma non è capace di generalizzare sui dati non visti.

L'obiettivo è quello di ridurre al massimo il gap tra overfittine e underfitting, una possibile risoluzione è quello di diminuire i parametri del modello, posso dare più dati o posso costringere il modello a scegliere su un sottoinsieme di parametri utilizzati. Queste tecniche vengono chiamate regolarizzazione.

Che tipo di dati utilizzeremo, dei dati di tipo tabellare. Gli algoritmo che vediamo, nella maggior parte dei casi, non partono dal dato grezzo (immagine RGB) ma partono da delle caratteristiche di questi dati grezzi estratti da altri algoritmi, e trasformano l'entità grezza in un vettore di caratteristiche. Ogni entità che sarà sottoposta al modello sarà un vettore. Ottengo quindi da tanti dati una matrice di caratteristiche. Ogni riga della matrice definisce un istanza.

Oltre a questa matrice abbiamo anche un vettore delle classi (in caso di apprendimento supervisionato), solitamente indicato con  $y$ , che per ogni istanza dice la classe.



Il processo di training è il processo di apprendimento dei parametri del modello. La performance, che viene chiamata *lost function*, è la funzione che guida il processo di apprendimento, perchè ci dice se stiamo facendo bene o male, perchè nella maggior parte dei casi fare male viene sfruttato per migliorare i parametri in modo tale che alla prossima iterazione mi comporti meglio. In modo tale da ridurre sempre di più la perdita.

| Term             | Meaning                                                     | Synonyms                                                                        |
|------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------|
| Training example | Riga della feature matrix. Features associate ad un oggetto | Osservazione, record, istanza, campione                                         |
| Training         | Model fitting                                               |                                                                                 |
| Feature $x$      | Colonna nella feature matrix.                               | Predictor, variabile, input, attributo, covariato                               |
| Target $y$       | Classe, Colonna delle etichette                             | Esito, output, response variable, variabile dipendente, etichetta, ground truth |
| Loss function    | Funzione obiettivo che misura la performance del modello    | Cost function, error function,                                                  |

## Logistic Regression (model-based)

Utilizza classificazione binaria, il che vuol dire che partiamo da un istanza che indicheremo con  $x$  segnato ed ogni elemento del vettore corrisponde ad una feature diversa. Ogni etichetta sarà 1 (istanza positiva) o 0 (istanza negativa). E' un approccio model base, il modello è di tipo lineare. Siamo capaci solo di tirare delle rette. Vogliamo trovare la probabilità che l'istanza appartenga ad una classe data una condizione. Quindi il modello dato un input restituisce

isce la PROBABILITA' che DATO l'input la sua classe o etichetta sia 1, la formulazione è questa:

$$P(y^i = 1|x^i) = P^i$$

Come calcoliamo la probabilità?

Odds: la odds di una probabilità è la probabilità fratto la probabilità opposta:

$$\text{odds}(p) = \frac{p}{p-1}$$

voglio che il valore di un evento con probabilità 0 vada a meno infinito:

$$\text{logit}(p) = \ln \frac{p}{p-1}$$

rimane però la logica della probabilità :

$$if(p_1 > p_2) \{ \text{logit}(p_1) > \text{logit}(p_2) \}$$

Per ora stiamo solo manipolando una probabilità, ma non sappiamo trovarla, dobbiamo quindi crearne un'altra partendo dalla logit, si chiama z:

$$z = w_1x_1 + w_2x_2 + \dots + w_mx_m + b$$

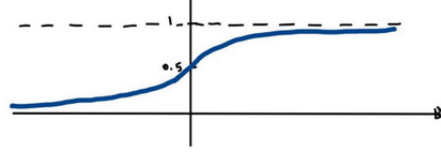
Tutti gli x sono associati alla probabilità ed al dato che ci è stato fornito, non possiamo modificarli, invece i parametri x e b fanno parte del modello e possiamo modificarli in base all'input.

L'algoritmo di logistic regression identifica i valori da dare ai parametri, tali per cui la probabilità di vedere il dataset che abbiamo è massima. Impariamo quindi dei pesi da associare alle singole colonne, il peso mi dice quanto è importante la colonna. Quello che facciamo quindi è cambiare i valori minimizzando la funzione di perdita o massimizzando la funzione di costo.

La parte di learning diventa una parte di modifica di parametri del modello per aumentare le performance.

Adesso dobbiamo fare un'inversione di variabile, l'obiettivo è avere qualcosa che dica cos'è p, non il logit di p. Per portare fuori la p applico la funzione esponenziale (inversa del logaritmo) ad entrambi i membri dell'uguaglianza. Quindi ora che p non è più nel logaritmo applichiamo una serie di operazioni fino ad ottenere che p è uguale a qualcosa che dipende solo da z. Otteniamo la funzione sigmoide.

$$\begin{aligned} \text{logit}(p) = \ln \frac{p}{1-p} = z &\rightarrow e^z = e^{\ln \frac{p}{1-p}} \rightarrow (1-p)e^z = p \rightarrow e^z - pe^z = p \rightarrow e^z = p + pe^z \\ &\rightarrow e^z = p(1 + e^z) \rightarrow \frac{1}{1 + e^{-z}} = \sigma(z) = \sigma(w^T x^i + b) \end{aligned}$$



La formula: la probabilità che all'istanza  $x_i$  venga assegnata una caratteristica  $y_i$ , dati i parametri del modello (posso considerarli fissi).

$$P(y^i = 1 | x_j^i w_i b) = \sigma(w^T x^i + b) = \frac{1}{1 + e^{-w^T x^i + b}}$$

Quindi in generale la probabilità che l'entità  $x$  abbia la caratteristica  $y$  può essere 0 o 1, dato che a livello matematico è scomodo la riscriviamo così:

$$\sigma(z)^{y^i} (1 - \sigma(z))^{1-y^i}$$

Ora dobbiamo fargli apprendere qualcosa, dobbiamo trovare i valori di  $b$  e  $w$  che massimizzano la performance. Come calcolo la performance? Se faccio vedere al mio modello tutta la feature matrix che probabilità ho che il modello generi il vettore delle labels? Quindi la massimizzazione dei pesi deve valere per tutto il dataset che ho visto, non può valere per una singola istanza.

$$P(y | M_i w_i b_i)?$$

Dobbiamo assumere che tutte le istanze della matrix siano indipendenti. Grazie a questa cosa diventa il prodotto delle probabilità di ogni evento, cosa che noi abbiamo già. In pratica ora quello che dobbiamo fare è trovare il massimo di questa funzione (fare la derivata), esistono tanti algoritmi che trovano il massimo.

$$\begin{aligned} P(y | M_i w_i b_i) &= P(y^1, \dots, y^n | x^1, \dots, x^n, w_i, b) \\ &= P\left(\bigwedge_{i=1}^n (y^i = 1 | x_j^i w_i b)\right) \\ &= \prod_{i=1}^n P(y^i = 1 | x_j^i w_i b) \\ &= \prod_{i=1}^n \sigma(z^i)^{y^i} (1 - \sigma(z^i))^{1-y^i} \end{aligned}$$

Per questioni di ottimizzazione è meglio passare a questi algoritmi non la produttoria ma la sommatoria. Faccio quindi il logaritmo della funzione, tanto non cambia dov'è il massimo, cambia solo il valore del massimo, che a noi non interessa.

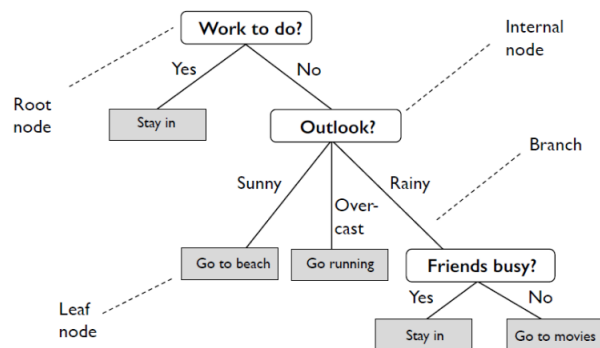
$$\sum_{i=1}^n y^i \log(\sigma(z^i)) + (1 - y^i) \log(1 - \sigma(z^i))$$

Quindi concludendo, l'algoritmo calcola le probabilità che un istanza appartenga ad una classe, poi prendiamo la classe che ha probabilità più alta. In caso di classificazione binaria lo standard è che se la probabilità è maggiore di 0.5 diventa 1 e se minore diventa 0. Possiamo modificare questa soglia.

Logistic regression impara i parametri della combinazione lineare (i pesi) più l'intercetta (b).

La cosa interessante è che se guardo il valore assoluto dei parametri questo mi dice quanto è importante quella particolare feature rispetto alle altre nella classificazione, quindi se vado a modificare anche poco una feature alla quale è associato un coefficiente molto elevato, questo avrà un impatto molto maggiore rispetto a modificare una feature con peso quasi zero.

## Alberi di decisione



Nell'ambito della classificazione ogni nodo corrisponde ad una feature ed i rami che escono dal nodo rappresentano i possibili valori che la feature ha. La classificazione avviene nei nodi foglia. Effettuo la classificazione seguendo l'albero a seconda dei valori delle feature, la classificazione avviene nei nodi foglia. Dal punto di vista del training dataset vuol dire che un albero di decisione man mano che arriva ad un nodo scinde il dataset di training in modo che alcuni campioni vadano in un ramo ed alcuni in un altro. Dal punto di vista teorico non facciamo altro che splittare in maniera gerarchica lo spazio delle feature in modo che ogni campione alla fine faccia parte di una sola feature.

Adesso dobbiamo fare l'apprendimento, ovvero come costruire l'albero di decisione a partire dal dataset di training. Considero una feature come nodo, splitto il dataset di training nei rami corrispondenti alla feature, vado avanti a fare questo splitting per ogni nodo fino a quando il nodo figlio si dice puro, significa che tutti i campioni del dataset di training che sono stati splittati in quel nodo appartengono tutti alla stessa classe.

- Seleziono una feature per fare lo split del nodo genitore
- Mi fermo se il nodo figlio è puro

- Torno allo step 1 per ognuno dei due nodi figli

Dato un insieme di feature posso costruire moltissimi alberi diversi, in base all'ordine in cui posiziono i nodi, le feature più in alto nell'albero sono più importanti nello splitting, quindi devo capire quali sono le feature che meglio classificano i campioni di training e metterli all'inizio dell'albero. Il concetto che si utilizza è quello dell' information gain, l'informazione nei nodi è la purezza ed il guadagno ce l'ho se nei nuovi nodi ho un incremento della purezza questa cosa ci ricorda l'entropia (l'incertezza che un elemento appartenga ad un nodo). Se vogliamo scriverla consideriamo una misura di impurità del nodo, per noi l'entropia, considero una feature  $f$  devo vedere se i nodi figli mi danno guadagno di informazione (sono più puri)

L'information gain è la misura dell'impurità del nodo padre meno la somma delle misure di impurità dei nodi figli:

$$IG(D_p, f) = I(D_p) - \sum_{j \in \text{values}(f)} \frac{N_j}{N_p} I(D_j)$$

- $I$ : è la funzione che misura l'impurità di un dato nodo (entropia)
- $f$ : è la feature
- $D_p$ : è il training set al nodo padre
- $D_j$ : è il dataset al nodo figlio dopo aver fatto lo split

L'entropia ha questa equazione:

$$I_H(D) = \sum_{i=1}^c -p_i \log_2 p_i$$

è la somma sul numero di classi  $d_i - p_i$  del logaritmo in base due di  $p_i$ , dove  $p_i$  sono la frazione di campioni in questo dataset etichettati secondo la classe dell'indice.

- $D$ : il dataset
- $c$ : numero delle classi (lables)
- $p_i$ : frazione ( $> 0$ ) dei campioni in  $D$  etichettati  $i$  (i campioni che appartengono alla classe  $i$ )

L'entropia è minima se tutti i campioni appartengono alla stessa classe, nel caso binario l'entropia è :

$$-p_+ \log_2 p_+ - p_- \log_2 p_-$$

ed è 0, se invece ho incertezza massima (metà e metà dei campioni) l'entropia è massima.

Quindi ad ogni nodo calcolo l'information gain per ogni feature, seleziono per questo nodo la feature che mi da maggior information gain:

$$IG(D_p, f) = \text{Entropy}(D_p) - \sum_{j \in \text{values}(f)} \frac{N_j}{N_p} \text{Entropy}(D_j)$$

in caso di classificazione binaria:

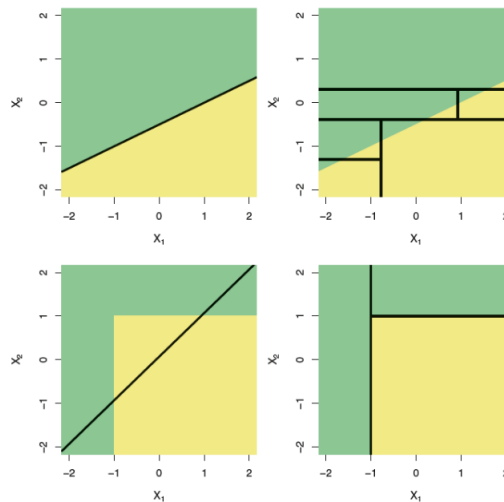
$$IG(D_p, f) = I_H(D_p) - \frac{N_{\text{left}}}{N_p} I_H(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I_H(D_{\text{right}})$$

### Pro e contro

Sono belli perchè sono facili da implementare, non è altro che la simulazione del nostro ragionamento, questo vuol dire che in un progetto di machine learning presentando il risultato con un albero di decisione guardando l'albero si capisce immediatamente come ha funzionato l'algoritmo. Inoltre sappiamo che tanto più una feature è in alto tanto più la feature è importante nella classificazione, quindi ci restituisce immediatamente l'importanza delle features. Ricordandoci che è un algoritmo greedy, potrebbe essere che scegliendo una feature così la combinazione potrebbe non essere la migliore. Non posso però dire che una feature che sta in basso sia non importante. E' indipendente dallo scaling delle feature, le features che stanno su scale diverse portano l'algoritmo a non considerarne alcune, quindi si portano tutte le feature in una scala 0-1, mentre l'albero di decisione è indipendente dalla scala delle feature, perchè fa il calcolo dell'IG senza guardare la scala delle altre feature. Gli alberi tendono a fare overfitting, perchè tipicamente si usano tantissime feature, siamo quindi in grado di costruire alberi con una profondità molto alta, più splitto più vado in fondo all'overfitting, dobbiamo avere delle tecniche per evitare l'overfitting, tipicamente si fa costruire tutto l'albero e poi si taglia l'albero (pruning). Gli alberi tipicamente sono molto sensibili al cambiamento dei dati.

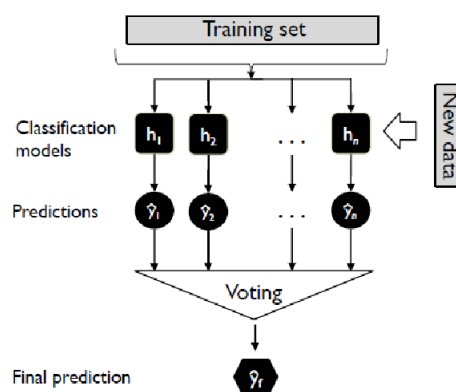
Gli alberi di decisione presi insieme attualmente sembra che siano (a parte il deep learning) l'algoritmo che funziona meglio.

In questo esempio vediamo come un albero di decisione non può dividere lo spazio delle feature in modo lineare, ma si deve adattare e farlo a "scalini".

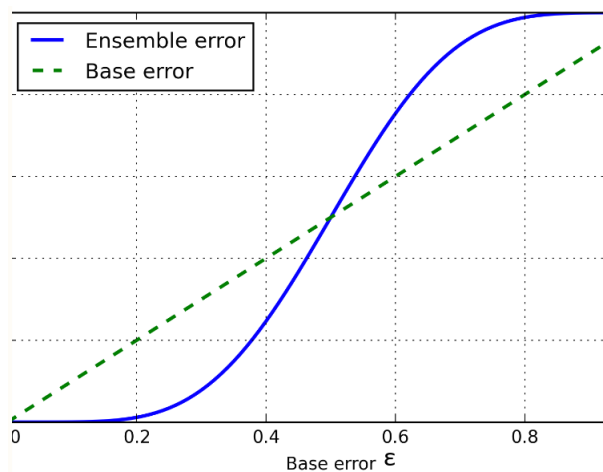


## Random Forest

E' un metodo ensemble, sono metodi basati sull'idea che invece che usare un unico algoritmo uso tanti algoritmi, ne metto insieme le previsioni, con l'idea che se uso più alberi ottengo un'accuratezza della previsione maggiore che usarne 1. Tipicamente abbiamo un training dataset, addestro tanti alberi, poi ho un nuovo campione di cui fare la predizione, ogni albero darà una predizione diversa, dico che la predizione finale è la classe che ha ricevuto più voti. Questo è un meta classificatore. Funziona molto bene perchè si dimostra che un ensemble di metodi ha sempre delle performance migliori sul singolo metodo.



Il problema però ora è come costruire i vari alberi di decisione per averli diversi.



## Bagging

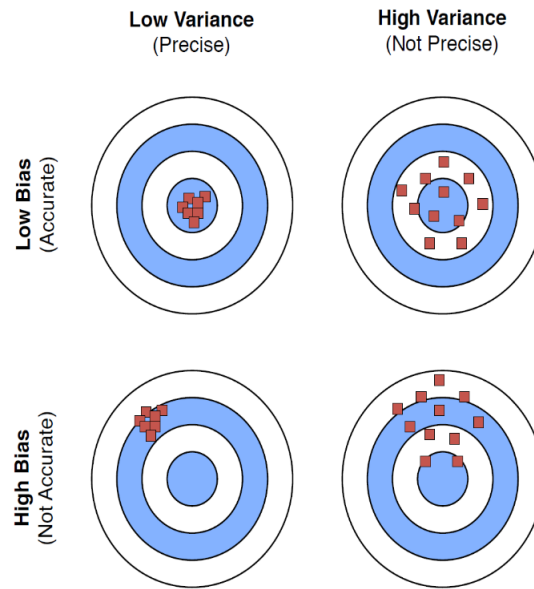
L'idea è questa, non cambio come costruisco l'albero, ma uso dei sotto campioni diversi del dataset di training. Ad ogni albero do un sotto campione diverso del dataset di training. In particolare si usano i campioni bootstrap, quando voglio ricampionare un insieme di campioni posso utilizzare una procedura bootstrap, prendo il set di campioni, ne estraggo 1 e poi lo rimetto dentro e rifaccio un'altra estrazione, quindi estraggo in modo random  $n$  volte tante quanto il numero di campioni ogni volta riconsiderandoli tutti.

Let  $n$  be the number of bootstrap samples

```
for i=1 to n do
  Draw bootstrap sample of size  $m$ , Diffusione
  Train base classifier  $h_i$  on  $D_i$ 
```

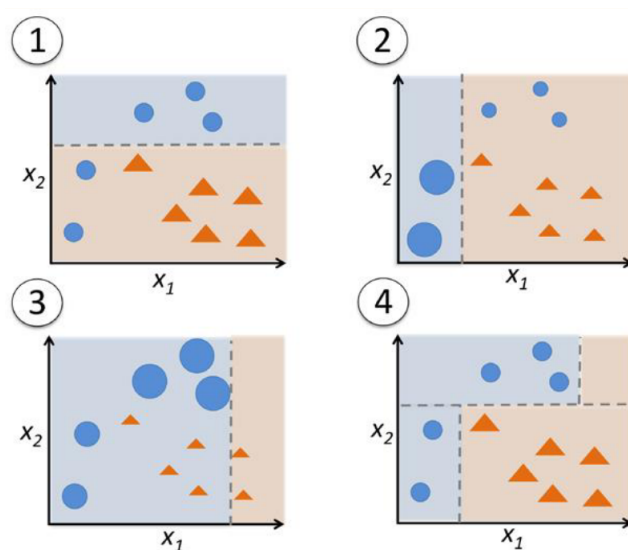
Nelle random forest utilizzo più alberi, in cui l'algoritmo di costruzione degli alberi è lo stesso, ma ogni albero è trainato con un set che è una copia bootstrap del dataset di training originale.

Lo faccio utilizzando metodi weak learning, metodi che hanno alta varianza. In pratica utilizzare il Bagging permette di ridurre la varianza di un metodo di apprendimento statistico.



## Boosting

Il Boosting è un altro metodo per creare i vari alberi, dove il training set è riscritto ad ogni iterazione, basato sugli errori precedenti. In pratica costruisce gli alberi con il bagging e poi va a correggere gli errori di classificazione.



## Random Forest

Mette insieme vari approcci, pare da alberi costruiti con il bagging ed ad ogni nodo carica un sottoinsieme delle feature andando a creare alberi molto diversi tra di loro. E' il metodo di machine learning (a parte il deep learning) che tipicamente ha performance migliori:

- Facile
- Non soffre dello scaling
- Ha ottime performance

Devo però prima della costruzione scegliere gli iper-parametri (i parametri che scelgo a priori). Inoltre durante la costruzione degli alberi non li costruiamo troppo profondi, rimaniamo lontani dall'overfitting evitando il pruning.

# Information Diffusion

La diffusione delle informazioni è un tema molto studiato in sociologia, noi lo affrontiamo dal punto di vista dei social media online. In generale è il processo per cui un pezzo di informazione viene diffuso ed attraverso le interazioni tra gli attori del mezzo raggiunge un set di individui.

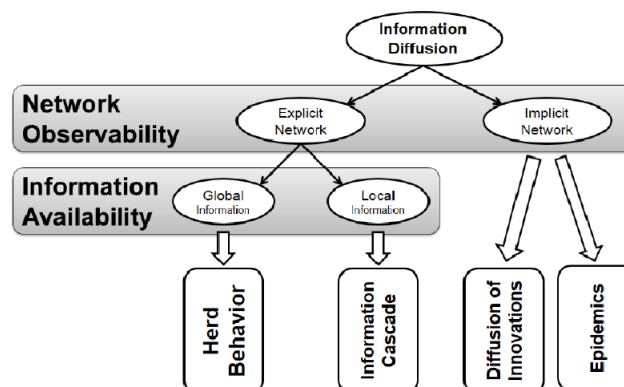
Elementi:

- Mittenti dell'informazione, può essere 1 o molti (piccolo sottoinsieme del sistema)
- Reciver: chi riceve l'informazione alla fine del processo di informazione, sui social chiaramente vogliamo che questo numero sia molto più grande dei mittenti
- Medium: per noi la piattaforma social

Esiste una classificazione dei processi di informazione in base a due parametri:

- Osservabilità della rete:
  - Rete esplicita: social network, i contatti tra due nodi sono espliciti
  - Rete implicita: pandemia, nessuno conosce i contatti che occorrono tra due persone che contraggono la malattia
- Disponibilità informazione:
  - Informazione globale
  - Informazione locale

Si chiamano processi di interferenza, quei processi che tentano di massimizzare oppure minimizzare, rallentare, stoppare, il processo di diffusione.



## Processi di diffusione su rete esplicita

Herd Behavior, sono processi dove la rete è esplicita ed ogni nodo conosce tutta la rete.

Information cascade, sono processi dove la rete è esplicita ed ogni nodo conosce solo una porzione della rete.

### Heard Behaviour

Comportamento a gregge, conosciamo la rete e tutto è pubblico. Quando tutti gli individui del sistema tendono ad avere lo stesso comportamento. Nel mondo web si osserva tipicamente nelle aste online, perchè gli altri vedendo che si alzano le puntate puntano sullo stesso oggetto con l'idea che vale la pena puntare su quell'oggetto. Un comportamento simile avviene con le criptovalute, se un influencer le compra e lo dice c'è una buona probabilità di perdita.

### Information cascade

Si chiama cascata perchè quando un nodo pubblica un'informazione arriva ai suoi vicini che può decidere se diffonderla, questo vuol dire che il processo diffusivo è locale, perchè ogni nodo conosce solo i suoi vicini.

La rete deve essere un grafo orientato, perchè devo sapere da dove parte il messaggio e dove arriva. La decisione che un nodo può prendere quando un nodo riceve l'informazione è binaria, un nodo è attivato quando riceve l'informazione, una volta attivato può decidere se continuare a diffonderla o fermarla. Si assume anche che sia un processo progressivo, un nodo può diventare da inattivo ad attivo, ma non viceversa (se la diffusione è l'acquisto di un oggetto non ho il reso). Queste sono assunzioni che si fanno per modellare questo sistema.

Il modello più utilizzato per questa diffusione è l'Independent Cascade Model, che è centrato sul mittente dell'informazione, pensa il processo dal punto di vista del nodo che manda l'informazione (mittente centrico). Esiste un altro modello che è chiamato linear threshold model, che si concentra su chi riceve l'informazione.

Un nodo viene attivato al tempo  $t$  e può diffondere l'informazione solo al tempo  $t+1$ , è un'assunzione forte, ma va bene così perchè abbiamo una rete temporale. Si modella la diffusione dell'informazione ai vicini del nodo  $v$  attivato attraverso una probabilità, assegno ad un vicino del nodo  $v$  una probabilità che venga attivato (sempre al tempo  $t+1$ ). Si inizia con un set di nodi che ha l'informazione e per ogni step dell'algoritmo, ogni nodo che è stato attivato al tempo  $t$  vede se attiva i suoi vicini con una certa probabilità al tempo  $t+1$ , i nodi che al tempo  $t+1$  ricevono l'informazione al tempo  $t+2$  a loro volta provano a mandare l'informazione ai vicini che anche loro verranno attivati con una certa probabilità. Questo processo stocastico si ferma quando non ho più probabilità di mandare l'informazione.

## Processi di diffusione su rete implicita

Diffusione di innovazione: viene osservato il rate di accrescimento dell'informazione, non vediamo la rete.

Epidemic mode: la differenza è che nel modello epidemico l'individuo non ha possibilità di scelta, mentre negli altri un individuo può scegliere se adottare l'informazione o meno.

Cominciamo a prendere confidenza con l'ambiente di sviluppo, [deep note](#), è una macchina virtuale in cloud con 1/2 virtual core e 5Gb di ram. E' l'ambiente che utilizzeremo principalmente poi vedremo anche un ambiente locale. L'ambiente è python, nel caso servano dei moduli non contenuti nella distribuzione di python si possono caricare tramite file.

Nella parte di analisi del progetto non vuole vedere la distribuzione di un nodo e il confronto con reti casuali.

Cominciamo ad interfacciarci con le API di Twitter, che è un buon parco giochi per sperimentare con il social media Mining. Attraverso le twitter API abbiamo accesso alla piattaforma di Twitter, incluso i `twit`, `retwit`.

## Template SMM

Il processo di SMM è caratterizzato da 4 fasi:

- Analisi iniziale, seleziono il media ed effettuo un'analisi del problema:
  - Identificazione del problema
  - Individuare i dati, dalle piattaforme
  - Analisi di fattibilità. Ad esempio con Facebook ed Instagram possiamo agire solo sugli account che ci concedono i permessi. Noi usiamo Twitter perché ci permette di avere in lettura gli account seguiti ed i follower.
- Preparazione:
  - Software
  - Hardware
  - Redazione del diario del processo, mi scrivo cosa devo fare, qual'è la documentazione, com'è il processo di autenticazione
- Esecuzione:
  - Data extraction, scrivo il codice che si interfaccia con le API
  - Pre-processing e preparazione del dato
  - Applicazione algoritmi di mining
- Valutazione dei risultati

## Twitter

Nella duplicazione del notebook dobbiamo prendere un `accesstoken` dell'account Twitter, una volta che ce lo abbiamo dobbiamo inserirlo come variabile di ambiente del progetto `deepnote`. Una volta fatto questo dobbiamo ottenere un `account developer` di twitter ed ottenere un `access token`. Quello che andremo

a fare è commentare il template, che è quello che dobbiamo fare noi durante il lavoro di gruppo. Innanzitutto bisogna avere un'idea generale di come funziona il social media che stiamo analizzando, dal punto di vista dell'utente dobbiamo capire che caratteristiche possono interessare per il progetto, elementi e funzionalità che la piattaforma implementa rispetto agli altri. Successivamente scriviamo nome social media API per identificare l'API che il social espone e che ci serve per ottenere i social data. Una volta identificata l'API dobbiamo specificare l'endpoint dell'API stessa, sempre in caso di piattaforme molto diffuse ci sono diverse versioni delle API che corrispondono all'evoluzione degli endpoint rilasciati e supportati. Innanzitutto mettere un link se c'è un overview ad alto livello di come funziona l'architettura delle API, poi se ci sono eventuali esempi di progetti. Dopo che ho identificato i documenti nella documentazione che reputo di interesse, in particolare quei documenti che esprimono le entità in gioco che rappresentano la piattaforma (elementi della piattaforma modellati dalle API), vado a descrivere le entità dal punto di vista dell'account ed informazioni relative alle relazioni tra le entità, che si esprime all'interno dell'idea di network e grafo.

## Tweepy

E' una libreria python con cui si può interagire con le API della piattaforma

Una volta che ho le entità devo prenderle, presa la documentazione di un'entità ci restituisce dei metodi con una sintassi per prendere le entità, sono tutte restful API, sono quindi chiamate HTTP.

Per fare le richieste mette i parametri delle query in un dizionario, questo per usare il metodo di python unpacking:

```
query_dict = {'query': query}
```

```
risultati_ricerca = twitter_api_v2.search_recent_tweets(**query_dict)
#twitter_api_v2.search_recent_tweets(query = '#pechinoexpress')
```

Così facendo posso aggiungere tutti i parametri che voglio al dizionario e poi nella chiamata lo spacchetto e crea una variabile per ogni elemento del dizionario.

```
query_dict['end_time'] = 'oggi'
```

```
risultati_ricerca = twitter_api_v2.search_recent_tweets(**query_dict)
#twitter_api_v2.search_recent_tweets(query = '#pechinoexpress',
#end_time = 'oggi')
```

Tutta la storia delle pagine per avere tutti i twit, dobbiamo prendere dal campo meta della prima risposta e lo metto nella query della seconda e così via:

```

tweets = []
r = twitter_api_v2.search_recent_tweets(**query_dict)
next_token = r['meta'].get('next_token')
for t in r['data']:
    tweets.append(t['id'])
#finche' il next_token != None
while next_token:
    #Introduco un nuovo parametro next_token
    query_dict['next_token'] = next_token
    r = twitter_api_v2.search_recent_tweets(**query_dict)
    #Cambio il valore di next_token per avere la nuova pag
    next_token = r['meta'].get('next_token')
    print(next_token)
for t in r['data']:
    tweets.append(t['id'])
#Esecuzione in pausa, per problema rate limit
    time.sleep(1)

```

## Graph visualization

La prima network che guardiamo è la rete sociale di delfini.

La seconda network è di tratta aeree, vogliamo mettere i nodi secondo coordinate geografiche. Qui abbiamo impostato un layout che posiziona i nodi in base alle coordinate, poi vogliamo cambiare la dimensione del nodo in base a quanto è il suo grafo.

Nella terza network parliamo di grafi bipartiti, è un grafo con due set di nodi ben precisi, nel nostro caso abbiamo il set dei fumetti ed il set dei supereroi, la particolarità è che gli archi appaiono solo tra elementi di set diversi, vogliamo visualizzare i nodi di ogni set in fila